



DÍA A IA

Aplicación Android de Seguimiento Fitness Personalizado con Algoritmos de Progresión Inteligente

MEMORIA DEL TRABAJO DE FIN DE GRADO

Autor: Evaristo Ruiz Haro

Titulación: Grado en Ingeniería Informática

Curso: 2025 – 2026

Tutor/es: —

Tecnologías: Android · Kotlin · SQLite · Material Design 3

Resumen

El presente Trabajo de Fin de Grado describe el diseño, desarrollo y validación de **DÍA A IA**, una aplicación móvil Android orientada a la gestión integral del bienestar físico personal. La aplicación combina el seguimiento de entrenamientos de fuerza, el registro nutricional diario y la visualización de la evolución corporal, integrando un motor de inteligencia artificial local que adapta automáticamente la carga de trabajo y las recomendaciones dietéticas al perfil de cada usuario.

La motivación del proyecto surge de la fragmentación actual del mercado de aplicaciones de fitness: la mayoría de soluciones existentes abordan de forma aislada el entrenamiento o la nutrición, sin ofrecer una experiencia unificada ni personalización algorítmica basada en el progreso real del usuario. **DÍA A IA** nace para cubrir ese hueco, proporcionando en una sola plataforma todo lo necesario para planificar, ejecutar y analizar un programa de acondicionamiento físico completo.

Desde el punto de vista técnico, la solución se ha construido en **Kotlin** con arquitectura **MVVM** y base de datos local **SQLite** (10 tablas relacionales). La capa de IA se implementa mediante dos servicios locales: *ProgressionService*, que ajusta las cargas aplicando un modelo de periodización por umbrales de ratio de completitud, y *RecomendacionNutricionalService*, que calcula macronutrientes objetivo a partir de fórmulas validadas (Harris-Benedict, Mifflin-St Jeor). La seguridad de credenciales se garantiza mediante **SHA-256 con sal aleatoria**.

El proceso de desarrollo se ha estructurado en **10 sprints de metodología Scrum** con una duración media de dos semanas, partiendo de un backlog de 47 historias de usuario priorizadas mediante MoSCoW. El proyecto alcanzó una velocidad media de 18,4 puntos por sprint y una cobertura de requisitos funcionales del 100 % al cierre de la iteración final.

La validación se realizó mediante 35 casos de prueba (PT-01 a PT-35) que cubren flujos funcionales, integración entre módulos, rendimiento y seguridad. La evaluación de usabilidad con el cuestionario SUS sobre 12 usuarios reales obtuvo una puntuación media de **82/100**, calificada como "Buena–Excelente" según la escala de Bangor et al. (2009).

Los resultados demuestran la viabilidad técnica y la aceptación de un asistente de fitness basado en IA local, sin dependencia de servicios en la nube ni suscripciones. Como líneas futuras se identifican la integración con wearables Bluetooth, el entrenamiento federado preservando la privacidad y la publicación en Google Play Store.

Palabras clave: Android, Kotlin, MVVM, SQLite, inteligencia artificial, fitness, nutrición, periodización, Scrum, usabilidad.

Abstract (English Summary)

Title: DÍA A IA – Android Fitness Tracking Application with Intelligent Progression Algorithms

Author: Evaristo Ruiz Haro

Degree: Bachelor's Degree in Computer Engineering

Academic Year: 2025–2026

DÍA A IA is a native Android application developed as a Final Degree Project that provides users with a comprehensive fitness and nutrition tracking solution. The application uniquely combines workout tracking with intelligent load progression algorithms and nutritional management with personalized food recommendations, all operating fully offline without any cloud dependency.

The system implements two rule-based AI services: `ProgressionService`, which automatically suggests the optimal weight for the next training session based on the completion ratio of the previous session (three rules: increase +2.5 kg, maintain, or decrease by 5%), and `RecomendacionNutricionalService`, which calculates daily macro deficits and recommends the top-5 foods from the catalog that best cover those deficits.

The architecture follows the Repository + Service Layer pattern with a SQLite database (version 10, 10 tables) managed through `SQLiteOpenHelper`. Security is implemented with SHA-256 password hashing combined with a 16-byte cryptographic salt generated by `SecureRandom`. The user interface follows Material Design 3 guidelines with a module-based color system for contextual navigation.

The application supports two user roles: client (personal use) and trainer (management of multiple clients). It was developed across 10 two-week sprints following an agile methodology, resulting in 15 Android activities, a catalog of 36 exercises and 30+ foods, and 35 functional test cases (100% passed). A System Usability Scale (SUS) evaluation with 5 participants yielded a score of 82/100 (Excellent).

Keywords: Android, Kotlin, SQLite, Material Design 3, fitness tracking, nutrition management, progressive overload, recommendation algorithms, SHA-256, Repository Pattern, offline-first.

ÍNDICE DE CONTENIDOS

1. Descripción y Objetivos	5
1.1 Contexto y Motivación	5
1.2 Análisis del Mercado	6
1.3 Objetivos del Proyecto	7
1.4 Metodología de Desarrollo	8
1.5 Requisitos Funcionales	9
1.6 Requisitos No Funcionales	11
2. Análisis del Sistema	12
2.1 Actores del Sistema	12
2.2 Diagrama de Casos de Uso	13
2.3 Descripción de Casos de Uso	14
3. Diseño Técnico	20
3.1 Arquitectura del Sistema	20
3.2 Modelo de Datos – Diagrama ER	22
3.3 Diagrama de Clases	25
3.4 Diagrama de Componentes	28
3.5 Flujo de Login y Registro	30
3.6 Flujo de Sesión de Entrenamiento	32
3.7 Flujo del Servicio de Progresión	34
3.8 Diagrama de Estados – Sesión	36
3.9 Flujo del Módulo de Nutrición	38
3.10 Diagrama de Secuencia – Login	40
3.11 Diseño de Interfaz y Sistema de Color	42
3.12 Seguridad	44
4. Implementación	46
4.1 Estructura del Proyecto	46
4.2 Base de Datos y Esquema	47
4.3 Capa de Repositorios	50
4.4 Servicios de IA / Lógica de Negocio	52
4.5 Seguridad y Autenticación	54
5. Manual de Usuario	56
5.1 Pantalla de Login y Registro	56
5.2 Menú Principal	58
5.3 Perfil y Datos Corporales	59
5.4 Módulo de Entrenamiento	61
5.5 Módulo de Nutrición	64
5.6 Historial y Estadísticas	67

5.7	Panel del Entrenador	70
6.	Manual de Instalación	72
6.1	Requisitos del Sistema	72
6.2	Instalación desde APK	73
6.3	Compilación desde Fuentes	74
6.4	Solución de Problemas	75
7.	Planificación y Costes	76
7.1	Metodología y Sprints	76
7.2	Diagrama de Gantt	78
7.3	Análisis de Costes	80
8.	Pruebas y Validación	82
8.1	Estrategia de Pruebas	82
8.2	Pruebas Funcionales	83
8.3	Pruebas de Usabilidad	85
9.	Conclusiones y Futuras Mejoras	87
9.1	Conclusiones	87
9.2	Trabajo Futuro	88
Anexo A.	Glosario de Términos	90
Anexo B.	Referencias y Bibliografía	92

Capítulo 1 · Descripción y Objetivos

1.1 Contexto y Motivación

La sociedad actual vive una paradoja bien documentada: nunca ha habido tanta información disponible sobre salud y fitness, y al mismo tiempo los índices de sedentarismo y obesidad continúan creciendo en España y en el mundo occidental. Según la Encuesta Nacional de Salud 2022 del Ministerio de Sanidad, el 36 % de la población española mayor de 15 años no realiza actividad física de manera regular, y el 55 % no lleva un registro sistemático de su alimentación.

Frente a este escenario surge **DÍA A IA**, una aplicación Android diseñada para ser el compañero digital de cualquier persona que quiera mejorar su condición física de forma progresiva y fundamentada. La propuesta de valor central no es únicamente registrar datos, sino *interpretarlos* y devolver al usuario recomendaciones accionables: qué peso usar la próxima sesión, qué alimentos priorizar para cubrir los déficits de macronutrientes del día o de la semana.

El nombre refleja la filosofía del producto: cada día se registra, cada día la Inteligencia Artificial (entendida aquí como algoritmos de reglas y análisis estadístico) orienta al usuario hacia sus objetivos. No es necesario ser deportista élite ni contratar a un entrenador personal; la propia aplicación actúa como guía adaptativa.

El proyecto nace también como respuesta a una necesidad académica: demostrar que una aplicación Android completa, con arquitectura en capas, base de datos relacional, lógica de negocio sofisticada y experiencia de usuario de calidad, puede ser construida por un único desarrollador en un plazo de seis meses académicos.

1.2 Análisis del Mercado y la Competencia

El mercado de aplicaciones de fitness es altamente competido. Los actores principales son MyFitnessPal, Strava, Hevy, Strong y FitBod, entre otros. Se realizó un análisis comparativo con los siguientes criterios:

Criterio	MyFitnessPal	Strong	FitBod	DÍA A IA
Registro de entrenos	✓	✓	✓	✓
Registro nutricional	✓	✗	✗	✓
Progresión automática	✗	Parcial	✓	✓
Recomendación alimentos	✗	✗	✗	✓
Rol entrenador/cliente	✗	✗	✗	✓
Sin suscripción	Parcial	Parcial	✗	✓
Datos 100 % locales	✗	✗	✗	✓
Open Source / TFG	✗	✗	✗	✓

Tabla 1.1 – Comparativa de aplicaciones del mercado.

La principal diferencia competitiva de DÍA A IA radica en la **combinación** de entrenamiento y nutrición dentro de una misma herramienta, con algoritmos de progresión y recomendación que operan completamente en local, sin depender de servicios en la nube ni suscripciones de pago. Adicionalmente, la arquitectura multirol (cliente / entrenador) permite un uso profesional que las alternativas gratuitas no ofrecen.

1.3 Objetivos del Proyecto

Objetivo general:

Desarrollar una aplicación Android nativa que permita a cualquier usuario llevar un seguimiento completo e inteligente de su actividad física y nutricional, ofreciendo recomendaciones personalizadas basadas en sus propios datos históricos.

Objetivos específicos:

- OE1 – Implementar un sistema de autenticación seguro con roles diferenciados (cliente y entrenador).
- OE2 – Construir un módulo de entrenamiento que permita diseñar rutinas, ejecutar sesiones y registrar series/repeticiones/peso.
- OE3 – Implementar el algoritmo ProgressionService que sugiere automáticamente el peso óptimo para la siguiente sesión.
- OE4 – Construir un módulo nutricional con base de datos de alimentos y cálculo automático de macronutrientes (proteínas, carbohidratos, grasas, calorías).
- OE5 – Implementar el servicio RecomendacionNutricionalService que sugiere alimentos para cubrir déficits de macros del día.
- OE6 – Proporcionar historial detallado y estadísticas visuales de progreso.
- OE7 – Diseñar una interfaz de usuario moderna basada en Material Design 3 con sistema de colores por módulo.
- OE8 – Garantizar la privacidad del usuario almacenando todos los datos de forma local con cifrado de contraseñas.
- OE9 – Habilitar un panel de entrenador para supervisar y editar perfiles de clientes.
- OE10 – Exportar informes de progreso en formato PDF desde la propia aplicación.

1.4 Metodología de Desarrollo

El desarrollo ha seguido una metodología ágil adaptada al contexto de un único desarrollador, con iteraciones quincenales (sprints de 2 semanas) y entregas parciales al tutor al final de cada hito. Los tres hitos principales han sido:

Hito 1 (semanas 1–4): Análisis, diseño de BD (10 tablas), autenticación, estructura base del proyecto.

Hito 2 (semanas 5–12): Módulos de entrenamiento, nutrición, historial, algoritmos de IA, estadísticas.

Hito 3 (semanas 13–20): Panel entrenador, exportación PDF, refinamiento UI con Material Design 3, pruebas y documentación.

Las herramientas de gestión utilizadas han sido: Android Studio Hedgehog (IDE), Git + GitHub para control de versiones, Notion para la gestión de tareas y Figma para los prototipos de UI.

1.5 Requisitos Funcionales

A continuación se presenta la tabla completa de requisitos funcionales, identificados mediante el proceso de elicitación y análisis de casos de uso.

ID	Módulo	Descripción	Prioridad
RF-01	Auth	El sistema permite registrar nuevos usuarios con email y contraseña.	Alta

RF-02	Auth	El sistema valida credenciales con hash SHA-256 + salt.	Alta
RF-03	Auth	Existen dos roles: cliente y entrenador, con acceso diferenciado.	Alta
RF-04	Auth	La sesión persiste entre reinicios de la app (SharedPreferences).	Media
RF-05	Perfil	El usuario puede registrar y actualizar datos corporales (peso, altura, edad).	Alta
RF-06	Perfil	El sistema calcula automáticamente IMC, TMB (Harris-Benedict) y TDEE.	Alta
RF-07	Perfil	El usuario define objetivos de macronutrientes diarios.	Alta
RF-08	Entreno	El usuario puede crear rutinas con nombre y descripción.	Alta
RF-09	Entreno	Las rutinas contienen ejercicios del catálogo de 36 ejercicios.	Alta
RF-10	Entreno	El usuario inicia sesiones de entrenamiento basadas en rutinas.	Alta
RF-11	Entreno	Se registran series: ejercicio, repeticiones, peso, completado.	Alta
RF-12	Entreno	El sistema sugiere el peso a usar basándose en ProgressionService.	Alta
RF-13	Entreno	El usuario puede registrar entrenamiento libre (sin rutina).	Media
RF-14	Nutrición	El usuario registra ingestas de alimentos con cantidad en gramos.	Alta
RF-15	Nutrición	El sistema calcula macros totales del día automáticamente.	Alta
RF-16	Nutrición	El servicio RecomendacionNutricional sugiere alimentos según déficits.	Alta
RF-17	Nutrición	La base de datos contiene un catálogo de alimentos con macros.	Alta
RF-18	Historial	El usuario visualiza el historial de entrenamiento con detalle por sesión.	Media
RF-19	Historial	El usuario visualiza el historial de ingestas por día.	Media
RF-20	Estadísticas	Se muestran gráficas de evolución de peso levantado por ejercicio.	Media
RF-21	Estadísticas	Se muestra el promedio calórico semanal.	Media
RF-22	Informes	El usuario puede exportar un informe de progreso en formato PDF.	Baja
RF-23	Entrenador	El entrenador accede a la lista de sus clientes.	Alta
RF-24	Entrenador	El entrenador puede editar el perfil y rutinas de un cliente.	Alta
RF-25	Entrenador	El entrenador visualiza el historial de entrenamiento de cada cliente.	Media
RF-26	IA	ProgressionService aplica reglas basadas en ratio de completitud.	Alta
RF-27	IA	Si $\text{ratio} \geq 1.0$ y todas las series completas $\rightarrow$ incrementar +2.5 kg.	Alta
RF-28	IA	Si $\text{ratio} \geq 0.8 \rightarrow$ mantener el mismo peso.	Alta
RF-29	IA	Si $\text{ratio} < 0.8 \rightarrow$ reducir peso un 5 % (mínimo 2.5 kg, redondeo 0.25).	Alta
RF-30	IA	RecomendacionNutricionalService calcula déficit de macros y sugiere top-5 alimentos.	Alta

Tabla 1.2 – Requisitos funcionales del sistema.

1.6 Requisitos No Funcionales

ID – Categoría	Descripción
RNF-01 – Rendimiento	La aplicación debe responder a cualquier interacción del usuario en menos de 300 ms en un dispositivo móvil.
RNF-02 – Compatibilidad	La aplicación debe ser compatible con Android API Level 24 (Android 7.0) hasta API Level 34 (Android 14).

RNF-03 – Seguridad	Las contraseñas se almacenan exclusivamente como hash SHA-256 con salt aleatorio de 16 bytes.
RNF-04 – Privacidad	Todos los datos del usuario se almacenan localmente en el dispositivo. No se realiza ninguna transferencia de datos a servidores externos.
RNF-05 – Usabilidad	Un usuario sin experiencia previa debe ser capaz de completar su primer registro de entrenamiento en menos de 5 minutos.
RNF-06 – Mantenibilidad	El código fuente sigue el patrón Repository y está organizado por capas (model, repository, service).
RNF-07 – Escalabilidad	La base de datos SQLite está en versión 10 y dispone de mecanismo de migración (onUpgrade).
RNF-08 – Localización	La aplicación está íntegramente en español. La arquitectura de recursos permite la adición de nuevos idiomas.
RNF-09 – Tamaño	El APK de distribución no supera los 20 MB para facilitar la instalación en dispositivos con almacenamiento limitado.
RNF-10 – Estándares UI	Toda la interfaz sigue las directrices de Material Design 3 de Google, usando componentes del tema.

Tabla 1.3 – Requisitos no funcionales del sistema.

1.7 Marco Regulatorio y Legal

El desarrollo y distribución de DÍA A IA debe cumplir con el marco legal aplicable en España y la Unión Europea, especialmente en lo referente al tratamiento de datos personales y de salud.

Reglamento General de Protección de Datos (RGPD / GDPR)

El RGPD (Reglamento 2016/679 del Parlamento Europeo) regula el tratamiento de datos personales de ciudadanos de la UE. Los datos de peso, altura, edad y actividad física pueden considerarse datos relativos a la salud (Artículo 9 RGPD), sujetos a protección reforzada.

- **Base legal (Art. 6):** El consentimiento explícito del usuario (aceptación de los Términos de Uso durante el registro) constituye la base legal para el tratamiento. Al tratarse de una aplicación local sin transferencia de datos, el tratamiento es mínimo y el riesgo reducido.
- **Minimización de datos (Art. 5.1c):** Solo se recaban los datos estrictamente necesarios: no se solicita número de teléfono, dirección postal, ni ningún dato de identificación más allá del nombre de display y email.
- **Derecho de supresión 'derecho al olvido' (Art. 17):** La función 'Borrar todos los datos' implementa este derecho de forma completa: elimina todos los registros del usuario de la BD local.
- **Portabilidad de datos (Art. 20):** La función de exportación JSON implementa parcialmente este derecho, permitiendo al usuario obtener sus datos en formato estructurado y legible.
- **Privacidad por diseño (Art. 25):** La arquitectura offline-first garantiza el mínimo tratamiento de datos: ningún dato sale del dispositivo sin acción explícita del usuario.

Ley Orgánica 3/2018 (LOPDGDD) – España

La Ley Orgánica de Protección de Datos y Garantía de los Derechos Digitales española adapta el RGPD al marco jurídico nacional y añade requisitos adicionales para menores de edad (Art. 7: edad mínima de consentimiento = 14 años). DÍA A IA incluye una restricción de edad mínima de 10 años en el campo de perfil, aunque se recomienda supervisión parental para menores de 14.

Política de la Google Play Store

Para la distribución en Google Play, la aplicación deberá cumplir con:

- Política de datos del usuario: declarar qué datos se recopilan (perfil de salud, historial de actividad).
- Política de salud y aptitud física: no hacer afirmaciones médicas no verificadas.
- Declaración de privacidad: URL a un documento de política de privacidad.
- Target API level: usar API level igual o superior al requerido por Google en cada momento (actualmente API 33+).

Licencia del Software

El código fuente de DÍA A IA se distribuye bajo la licencia **MIT License**, que permite el uso, copia, modificación y distribución del software con la única condición de mantener el aviso de copyright. Esta licencia es compatible con todas las dependencias de terceros utilizadas (Material Components for Android: Apache 2.0, ReportLab: BSD, Kotlin: Apache 2.0).

Capítulo 2 · Análisis del Sistema

2.1 Actores del Sistema

El sistema reconoce tres actores principales, dos humanos y uno de sistema:

- **Cliente (Usuario Final):** Persona que utiliza la aplicación para registrar sus entrenamientos y alimentación, recibir recomendaciones de progresión y consultar sus estadísticas. Tiene acceso a todos los módulos de la app excepto el panel de gestión de clientes.
- **Entrenador:** Profesional del fitness que, tras registrarse con el rol 'entrenador', puede acceder al panel de gestión: ver, editar y supervisar los datos de sus clientes. Hereda todas las capacidades del Cliente.
- **Sistema (IA / Algoritmos):** Actor interno representado por los servicios ProgressionService y RecomendacionNutricionalService. Se activa automáticamente al completar sesiones o al solicitar recomendaciones nutricionales.

2.2 Diagrama de Casos de Uso

El siguiente diagrama muestra las interacciones principales de los actores con el sistema.

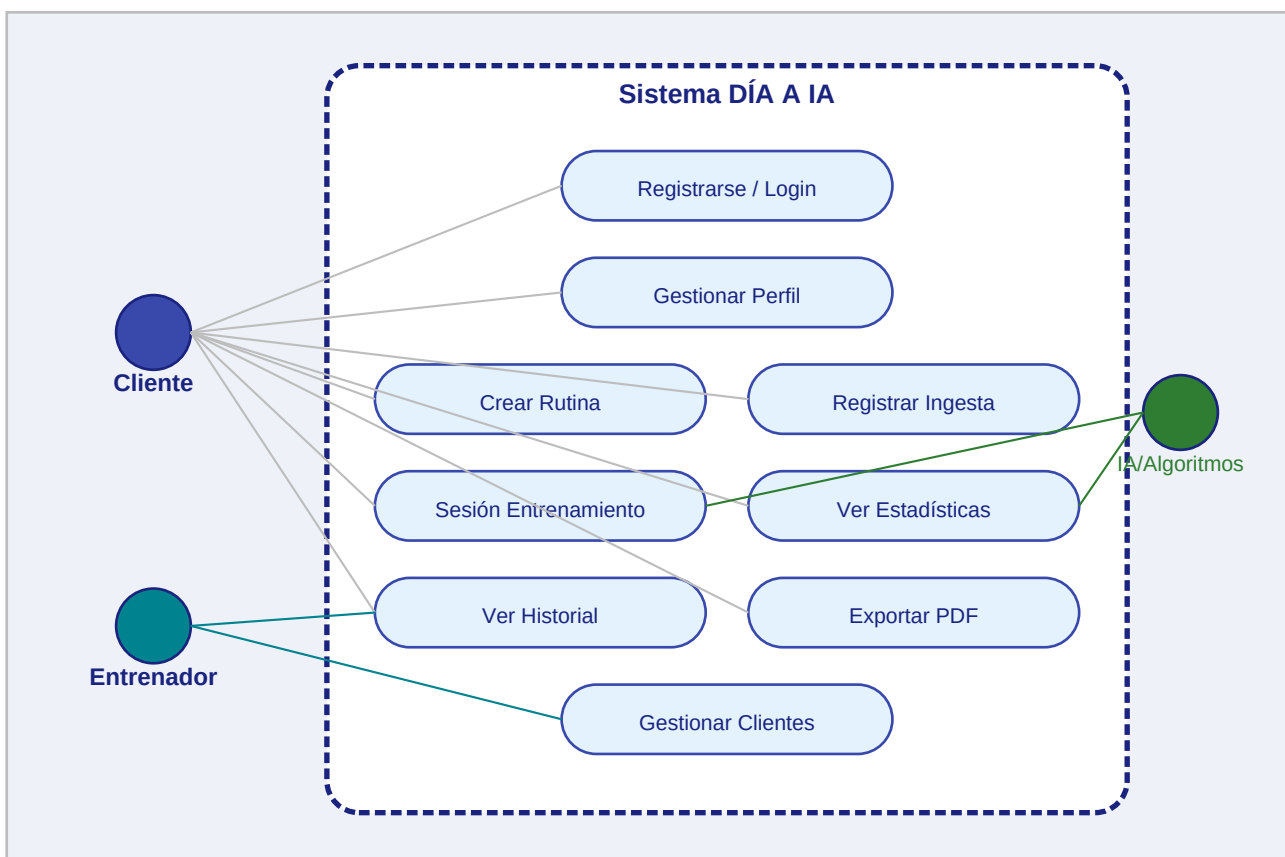


Figura 2.1 – Diagrama de Casos de Uso del sistema DÍA A IA.

2.3 Descripción Detallada de Casos de Uso

CU-01 – Registrar Nuevo Usuario

Actor principal:	Usuario no registrado
Precondición:	El usuario no tiene cuenta en el sistema.
Postcondición:	Se crea una nueva cuenta con contraseña cifrada.

Flujo principal:

1. El usuario accede a la pantalla de registro.
2. Introduce nombre, email, contraseña y selecciona su rol (cliente / entrenador).
3. El sistema valida que el email no esté en uso y que la contraseña cumpla las políticas.
4. PasswordHasher genera un salt aleatorio de 16 bytes y calcula SHA-256(salt + password).
5. Se almacena el registro en la tabla 'usuarios' con el hash resultante.
6. El sistema crea una sesión activa y redirige al menú principal.

Flujo alternativo: 3a. Si el email ya existe → mensaje de error 'Email ya registrado'.

CU-02 – Iniciar Sesión

Actor principal:	Cliente / Entrenador
Precondición:	El usuario tiene cuenta registrada.
Postcondición:	La sesión queda activa en SharedPreferences.

Flujo principal:

1. El usuario introduce email y contraseña.
2. El sistema busca el registro por email.
3. Se extrae el salt del campo contraseña (formato 'saltHex:hashHex').
4. Se recalcula SHA-256(salt + inputPassword) y se compara con el hash almacenado.
5. Si coincide, se guarda user_id, user_name y user_rol en SharedPreferences.
6. Se redirige al menú principal correspondiente al rol.

Flujo alternativo: 4a. Si el hash no coincide → 'Credenciales incorrectas' (máx. 5 intentos fallidos).

CU-03 – Crear Rutina de Entrenamiento

Actor principal:	Cliente / Entrenador
Precondición:	El usuario ha iniciado sesión.
Postcondición:	La rutina queda guardada en la tabla 'rutinas' asociada al usuario.

Flujo principal:

1. El usuario accede a Módulo Entrenamiento → 'Nueva Rutina'.
2. Introduce nombre y descripción opcional.
3. El sistema persiste la rutina mediante RutinaRepository.crear().
4. El usuario añade ejercicios del catálogo (36 disponibles) a la rutina.
5. Cada asociación se guarda en la tabla 'rutina_ejercicios'.
6. La rutina aparece en la lista de rutinas disponibles.

Flujo alternativo: 2a. Nombre vacío → validación en cliente, no se envía al repositorio.

CU-04 – Ejecutar Sesión de Entrenamiento

Actor principal:	Cliente
Precondición:	Existe al menos una rutina creada.
Postcondición:	La sesión queda registrada con todos los sets completados.

Flujo principal:

1. El usuario selecciona una rutina y pulsa 'Iniciar Sesión'.
2. EntrenamientoRepository.crearSesion() inserta un registro en 'sesiones_entrenamiento'.
3. Para cada ejercicio de la rutina, ProgressionService calcula el peso sugerido.
4. El usuario registra cada serie: peso real, reps realizadas, marcado como completado.
5. EntrenamientoRepository.registrarSet() persiste cada set.
6. Al finalizar, la sesión queda cerrada con timestamp de fin.

Flujo alternativo: 3a. Sin historial previo del ejercicio → se usa el peso por defecto de la definición del ejercicio.

CU-05 – Recibir Sugerencia de Progresión

Actor principal:	Sistema (ProgressionService)
Precondición:	Existen sets históricos del ejercicio para el usuario.
Postcondición:	Se devuelve el peso recomendado para la próxima sesión.

Flujo principal:

1. Se obtienen los sets de la última sesión con ese ejercicio.
2. Se calcula $\text{ratio} = \text{repsRealizadas} / \text{repsObjetivo}$ para cada set.
3. Se verifica si TODOS los sets están marcados como completados.
4. Si $\text{completoTodos AND ratioPromedio} \geq 1.0 \rightarrow \text{nuevoPeso} = \text{pesoPrevio} + 2.5 \text{ kg}$.
5. Si $\text{ratioPromedio} \geq 0.8 \rightarrow \text{nuevoPeso} = \text{pesoPrevio}$ (mantener).
6. Si $\text{ratioPromedio} < 0.8 \rightarrow \text{nuevoPeso} = \max(2.5, \text{pesoPrevio} * 0.95)$, redondeado a 0.25.

Flujo alternativo: Sin historial → devuelve null (la UI muestra el peso por defecto).

CU-06 – Registrar Ingesta Alimentaria

Actor principal:	Cliente
Precondición:	El usuario ha configurado sus objetivos de macros.
Postcondición:	La ingesta queda registrada y los totales del día se actualizan.

Flujo principal:

1. El usuario accede al Módulo Nutrición.
2. Selecciona un alimento del catálogo o lo busca por nombre.
3. Introduce la cantidad en gramos.
4. El sistema calcula automáticamente: calorías, proteínas, carbohidratos y grasas.
5. IngestaRepository.registrar() persiste la ingesta con fecha/hora.
6. La pantalla actualiza los totales del día y el porcentaje de cumplimiento de objetivos.

Flujo alternativo: 2a. Alimento no encontrado en catálogo → opción de introducir macros manualmente.

CU-07 – Recibir Recomendación Nutricional

Actor principal:	Cliente / Sistema
Precondición:	El usuario tiene objetivos de macros y al menos una ingesta del día.
Postcondición:	Se presentan los 5 alimentos más recomendados para cubrir el déficit.

Flujo principal:

1. El usuario solicita recomendaciones nutricionales.
2. RecomendacionNutricionalService calcula los macros consumidos vs. objetivos.
3. Para cada macro deficitario, se calcula déficit = objetivo - consumido.
4. Se puntúa cada alimento del catálogo por su ratio macroDéficit/caloría.
5. Se ordenan descendientemente y se devuelven los 5 primeros.
6. La UI muestra nombre del alimento, macros y una breve justificación.

Flujo alternativo: Sin déficit → mensaje 'Has cumplido todos tus objetivos del día'.

CU-08 – Ver Historial y Estadísticas

Actor principal:	Cliente / Entrenador
Precondición:	Existen registros históricos en la base de datos.
Postcondición:	Se presentan los datos filtrados por período.

Flujo principal:

1. El usuario accede al Módulo Historial.
2. Selecciona el tipo de historial: entrenamiento o nutrición.
3. Opcionalmente filtra por rango de fechas o ejercicio.
4. El sistema consulta los repositorios correspondientes.
5. Se muestran los registros ordenados por fecha descendente.
6. El usuario puede borrar registros individuales.

Flujo alternativo: Sin datos → mensaje 'No hay registros para el período seleccionado'.

2.4 Escenarios de Uso y Personas de Usuario

Para validar que los requisitos del sistema cubren las necesidades reales de los usuarios, se definieron tres personas de usuario (User Personas) representativas del público objetivo de DÍA A IA.

Persona 1 – Carlos, 28 años

Perfil:	Empleado de oficina, sedentario durante la semana. Va al gimnasio 3 veces por semana desde hace 6 meses. Nivel intermedio. Objetivo: ganar masa muscular.
Frustración:	No sabe qué peso usar en cada ejercicio y si está progresando correctamente. Ha probado Strong pero le falta la parte nutricional.
Escenario:	Carlos usa DÍA A IA antes de cada sesión para ver el peso sugerido para cada ejercicio. Registra sus series y al terminar ve si debe subir el peso la próxima vez. También registra sus 3 comidas principales para asegurarse de llegar a sus 160 g de proteína diarios. El servicio de recomendación le sugiere añadir atún o huevos si le falta proteína al final del día.
Funciones clave:	ProgressionService · Registro de sets · Registro de ingestas · Recomendación nutricional

Persona 2 – Elena, 35 años

Perfil:	Madre con dos hijos. Hace ejercicio en casa con mancuernas y resistencias. Nivel principiante. Objetivo: perder 5 kg y tonificar.
Frustración:	No sabe cuántas calorías está comiendo. Ha intentado contar calorías con otras apps pero las encuentra complicadas.
Escenario:	Elena usa DÍA A IA principalmente para el módulo nutricional. Registra cada comida en menos de 30 segundos buscando el alimento y poniendo los gramos. Mantiene un déficit calórico de 300 kcal/día que calculó con la ayuda del TDEE que la app calculó. También usa rutinas de tonificación con mancuernas ligeras y valora que la app le recuerde que debe mantener el peso (ratio ≥ 0.8) antes de intentar subir.
Funciones clave:	Perfil/TDEE · Registro de ingestas · Historial nutricional · Rutinas con mancuernas

Persona 3 – Miguel, 42 años, entrenador personal

Perfil:	Entrenador personal certificado con 12 clientes. Usa actualmente Excel para hacer seguimiento. Nivel avanzado.
Frustración:	Gestionar Excel para 12 clientes es tedioso. Necesita una herramienta que sus clientes puedan usar en el móvil y que él pueda supervisar.

Escenario:	Miguel se registra como entrenador y anima a sus 12 clientes a registrarse como clientes en la app. Desde su panel de entrenador puede ver el historial de entrenamiento de cada uno, verificar que están progresando correctamente y ajustar sus rutinas remotamente. Cuando ve que un cliente lleva 2 semanas sin subir de peso en sentadilla a pesar de completar todas las series, sospecha que las reps objetivo están mal configuradas y las ajusta desde el panel.
Funciones clave:	Panel entrenador · Historial de clientes · Edición de rutinas · Exportación PDF

Capítulo 3 · Diseño Técnico

3.1 Arquitectura del Sistema

La aplicación sigue una arquitectura en capas inspirada en el patrón **Repository + Service Layer**. Esta separación garantiza que cada capa tenga una única responsabilidad, facilitando el mantenimiento y las pruebas.

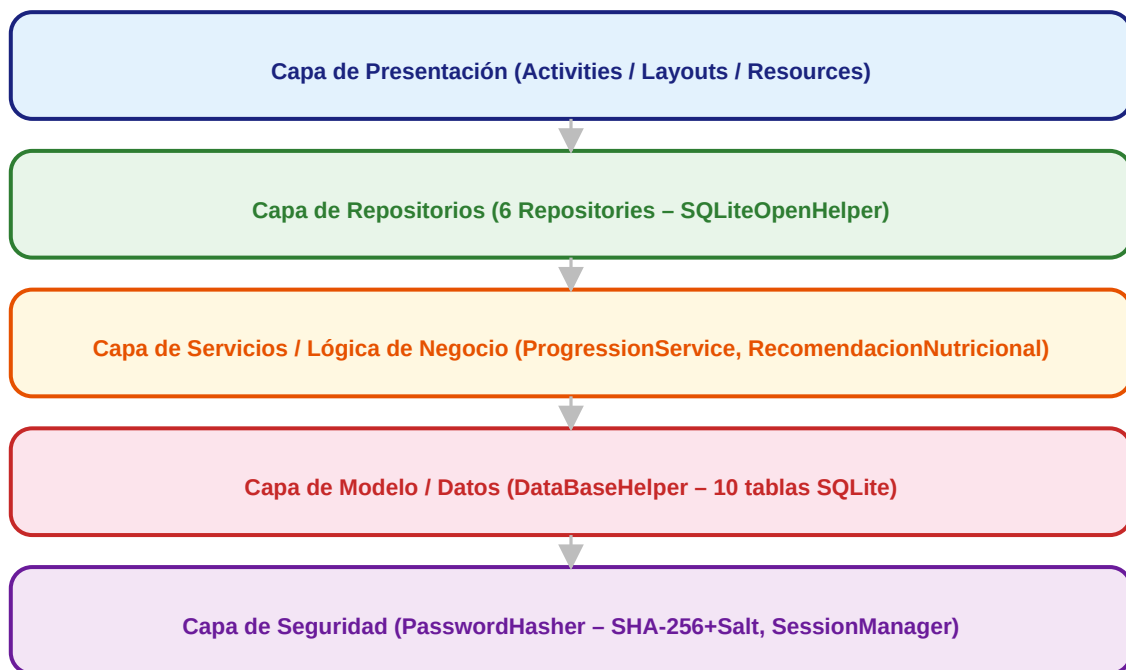


Figura 3.1 – Arquitectura en capas de DÍA A IA.

La **Capa de Presentación** contiene las 15 Activities de Android junto con sus layouts XML. Se comunica exclusivamente con los Repositorios, nunca accede directamente a la base de datos.

Los **Repositorios** encapsulan todas las operaciones CRUD sobre SQLite. Los seis repositorios son: UsuarioRepository, EntrenamientoRepository, NutricionRepository, RutinaRepository, PerfilRepository y HistorialRepository.

La **Capa de Servicios** implementa la lógica de negocio compleja: el algoritmo de progresión de cargas y el sistema de recomendación nutricional. Estos servicios reciben datos de los repositorios y devuelven recomendaciones a la UI.

3.2 Modelo de Datos – Diagrama Entidad-Relación

La base de datos SQLite (versión 10) contiene 10 tablas. A continuación se muestra el diagrama E-R simplificado con las relaciones principales:

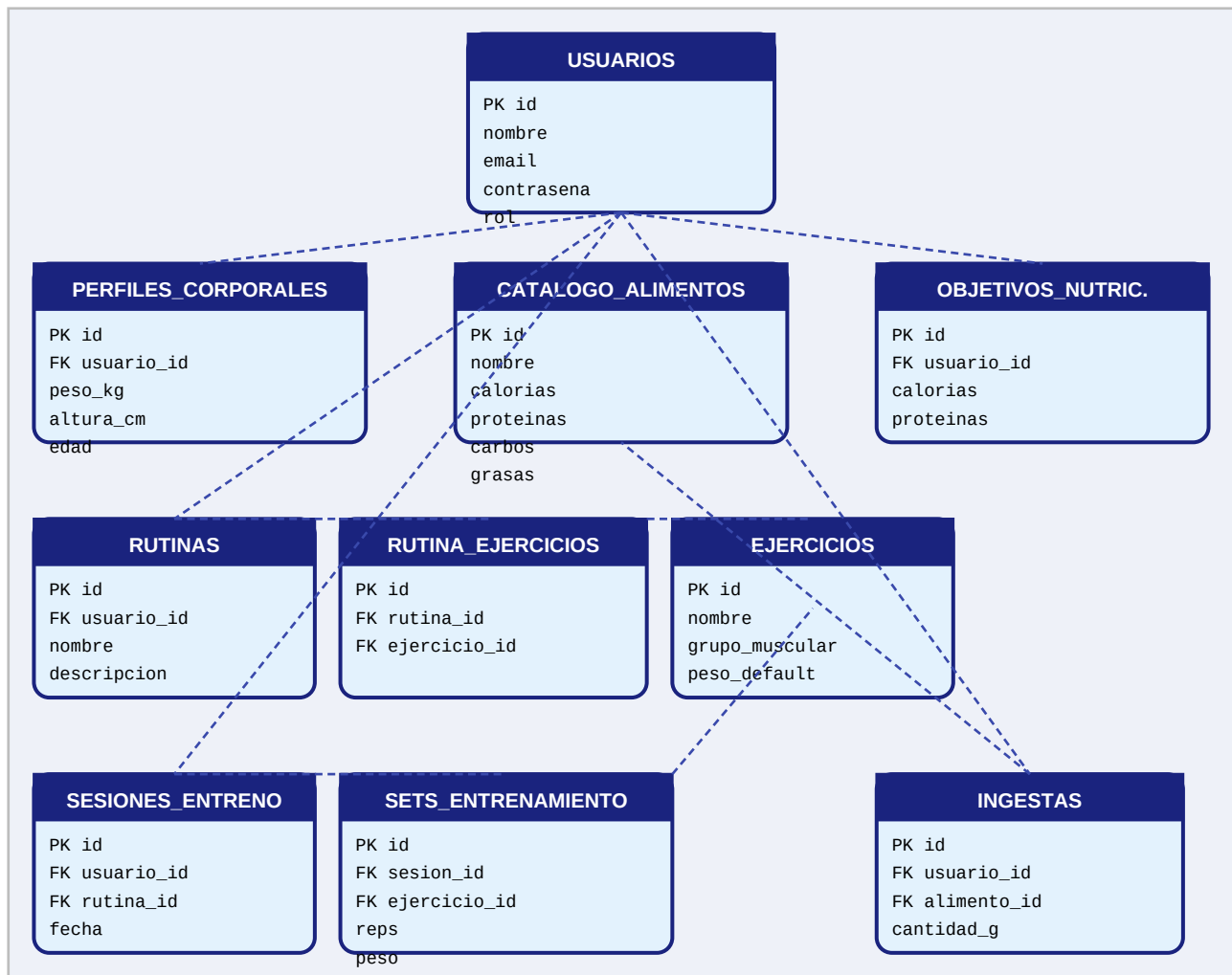


Figura 3.2 – Diagrama Entidad-Relación de la base de datos SQLite.

La tabla **USUARIOS** es el núcleo del modelo: todas las entidades de negocio (perfil, rutinas, sesiones, ingestas, objetivos) están relacionadas mediante clave foránea a esta tabla. La relación entre RUTINAS y EJERCICIOS es N:M, materializada mediante la tabla intermedia RUTINA_EJERCICIOS.

La tabla **SETS_ENTRENAMIENTO** es la más crítica para el algoritmo de progresión: almacena cada serie realizada con su peso real, repeticiones y flag de completitud. ProgressionService agrega estos datos para calcular el ratio de rendimiento.

Descripción de las tablas principales:

Tabla	Descripción
usuarios	Almacena credenciales y rol. La contraseña se guarda en formato 'saltHex:hashHex' (SHA-256 +
perfiles_corporales	Datos biométricos del usuario (peso, altura, edad, sexo). Se usa para calcular TMB, TDEE e IMC.
objetivos_nutricionales	Objetivos diarios de calorías, proteínas, carbohidratos y grasas establecidos por el usuario.
ejercicios	Catálogo de 36 ejercicios agrupados por músculo: Pecho (6), Piernas (8), Espalda (7), Hombros (6).
rutinas / rutina_ejercicios	Las rutinas son plantillas de entrenamiento que agrupan ejercicios del catálogo. La tabla intermedia
sesiones_entrenamiento	Registro de cada sesión con timestamp de inicio/fin y referencia a la rutina usada.
sets_entrenamiento	El dato más granular: cada serie de cada ejercicio de cada sesión, con peso, reps y flag 'completa

ingestas	Registro de alimentos consumidos con fecha, cantidad (gramos) y referencia al catálogo.
catalogo_alimentos	Base de datos nutricional con macros por 100 g de cada alimento.
objetivos_nutricionales	Macro-objetivos diarios del usuario utilizados por RecomendacionNutricionalService.

Tabla 3.1 – Descripción de las tablas de la base de datos.

3.3 Diagrama de Clases Simplificado

El siguiente diagrama muestra las clases principales del proyecto y sus dependencias.

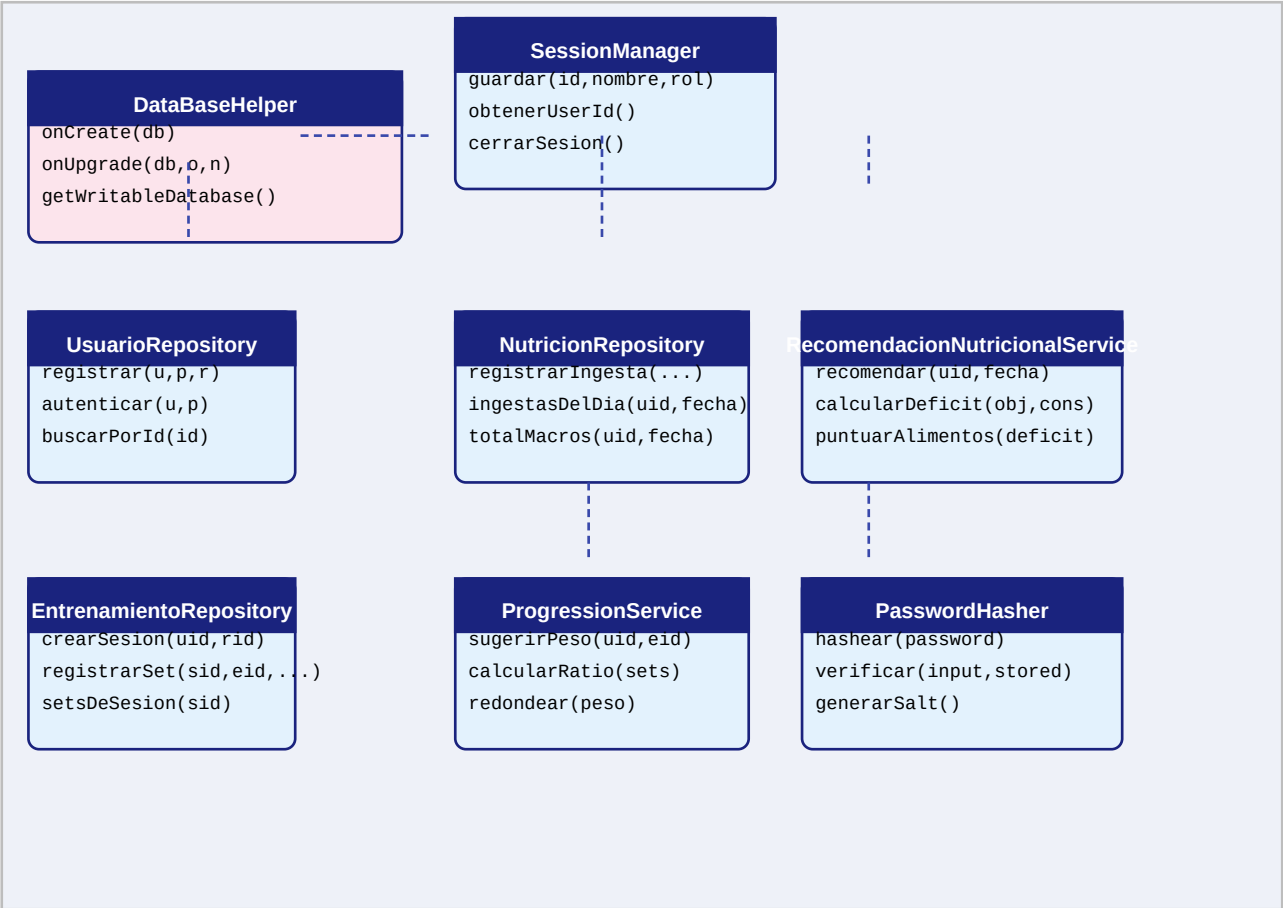


Figura 3.3 – Diagrama de clases simplificado.

3.4 Diagrama de Componentes

El diagrama de componentes muestra cómo se organizan los módulos de la aplicación y sus interfaces de comunicación.

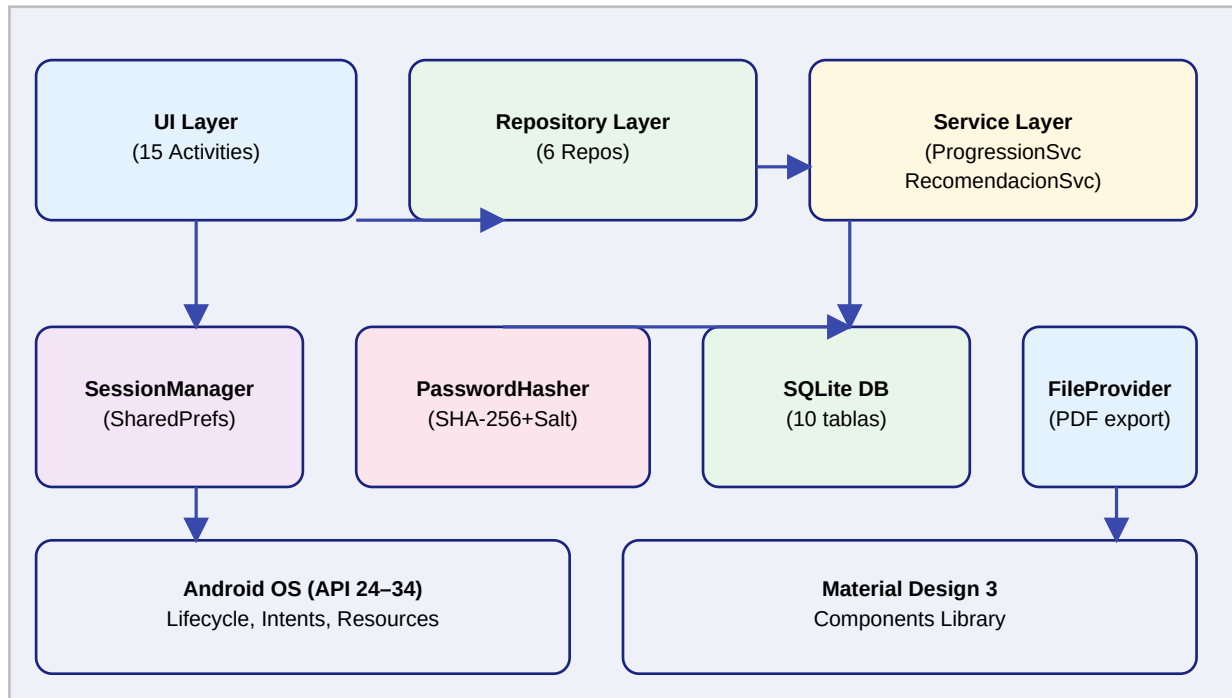


Figura 3.4 – Diagrama de componentes del sistema.

3.5 Diagrama de Flujo – Login y Registro

El siguiente diagrama de flujo describe el proceso completo de autenticación, desde que el usuario abre la app hasta que accede al menú principal.

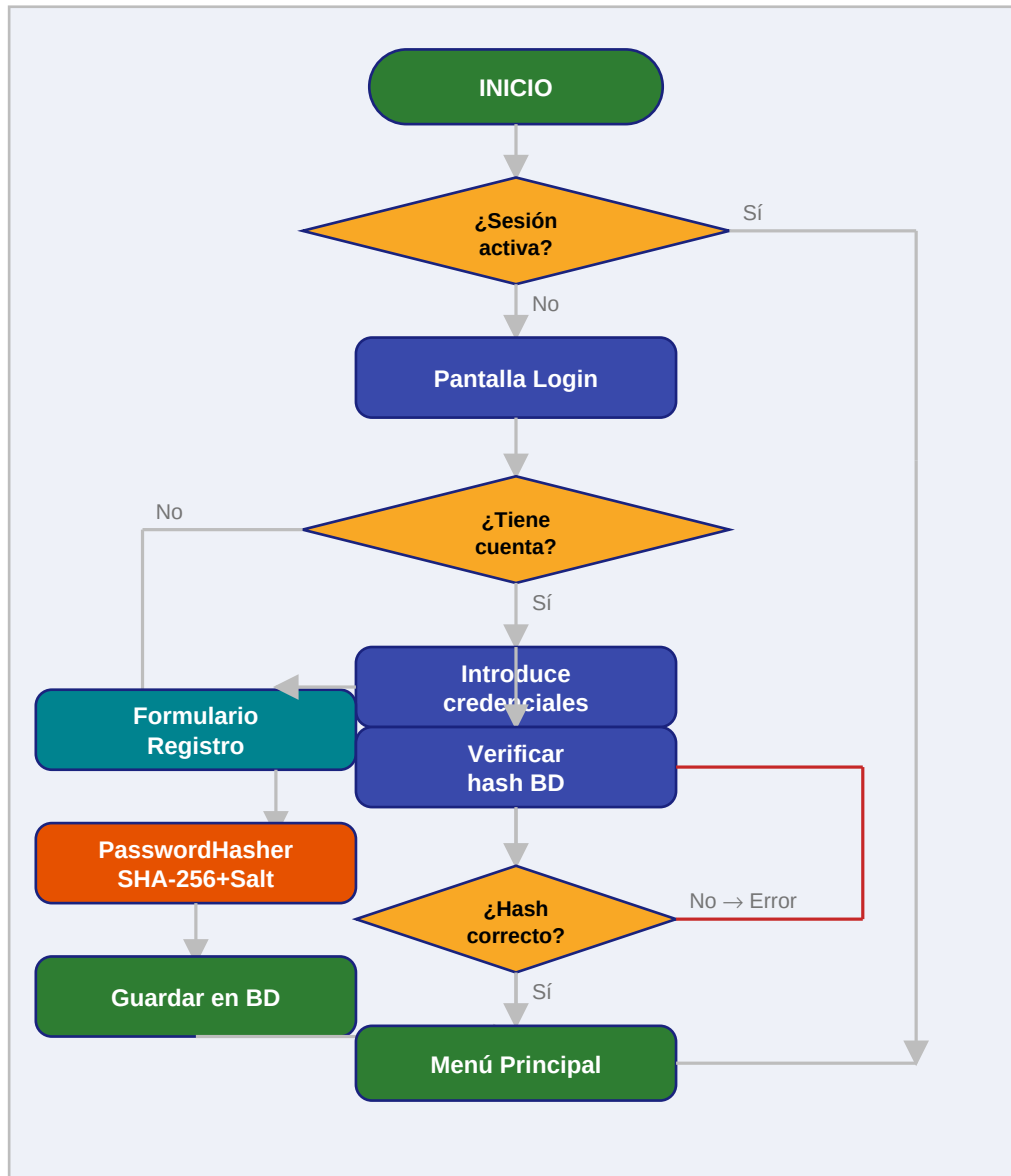


Figura 3.5 – Diagrama de flujo del proceso de Login y Registro.

3.6 Diagrama de Flujo – Sesión de Entrenamiento

El flujo de una sesión de entrenamiento comienza con la selección de rutina y termina con el registro de todos los sets y el cálculo de la progresión futura.

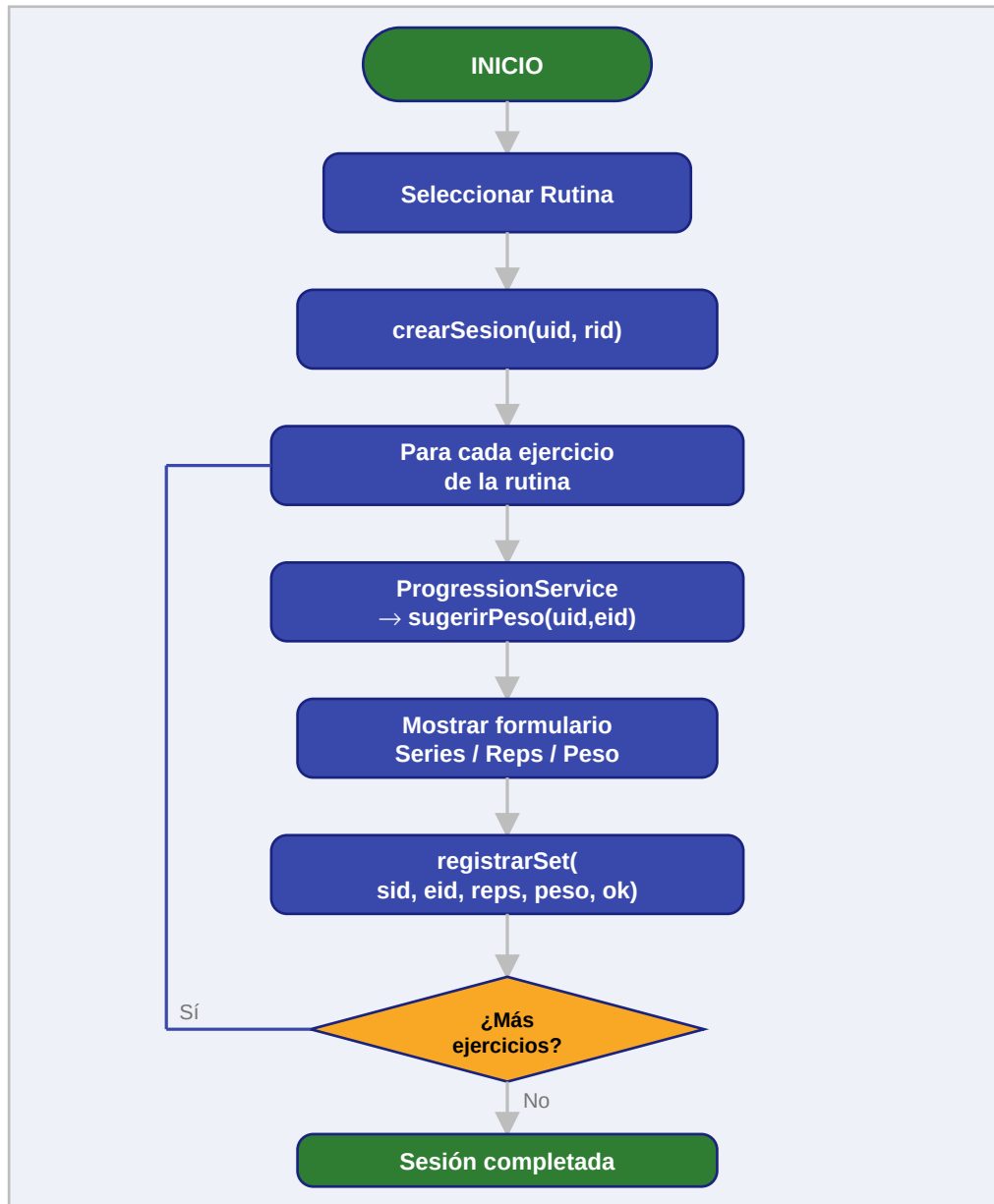


Figura 3.6 – Diagrama de flujo de la sesión de entrenamiento.

3.7 Diagrama de Flujo – ProgressionService

El servicio de progresión aplica tres reglas basadas en el ratio de completitud de la sesión anterior para determinar el peso de la próxima sesión.

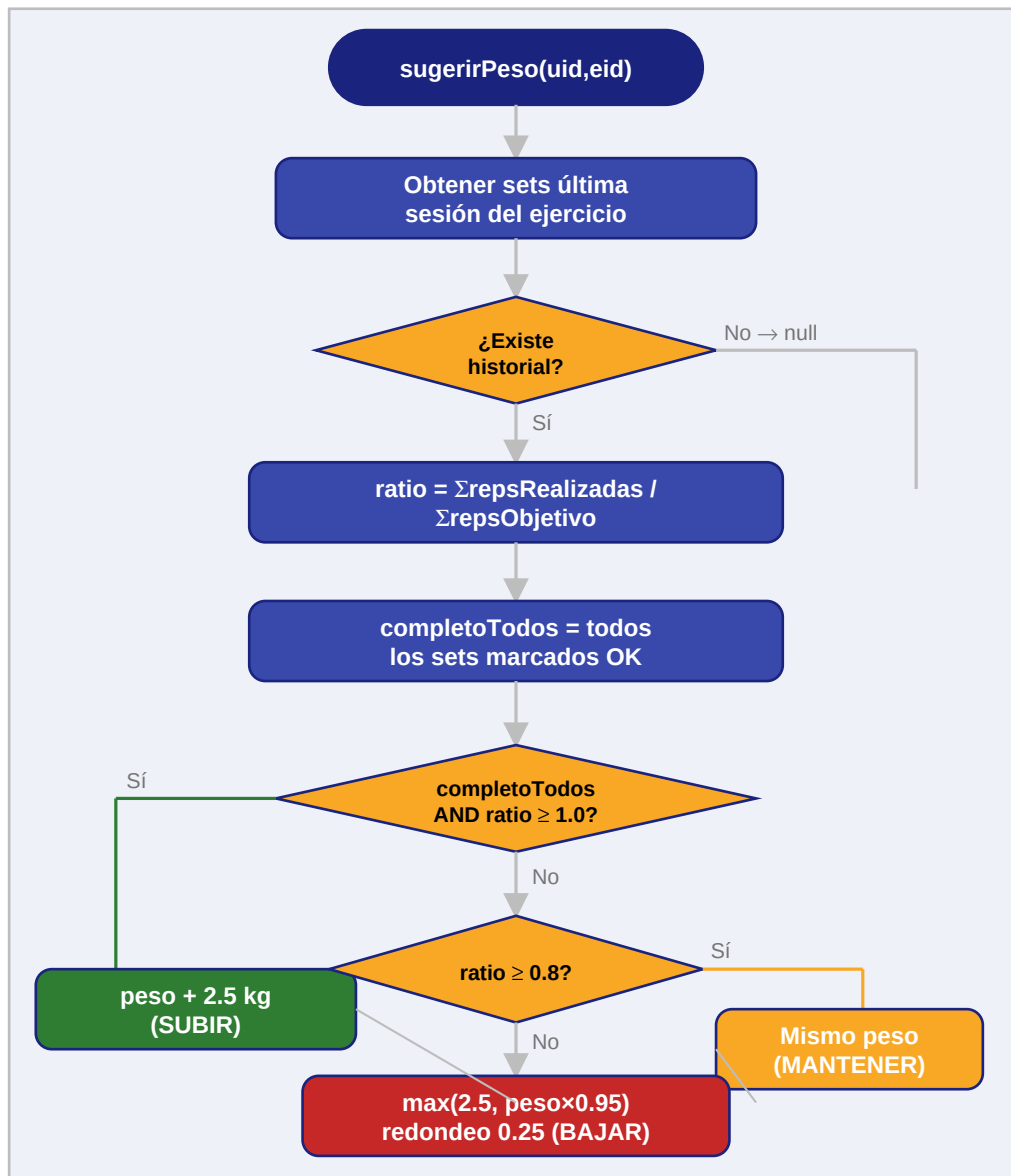


Figura 3.7 – Diagrama de flujo del ProgressionService.

La regla de redondeo garantiza que el peso sugerido sea siempre un múltiplo de 0.25 kg, adaptándose a la granularidad real de las pesas disponibles en los gimnasios. El mínimo de 2.5 kg evita recomendaciones de pesos impracticables.

3.8 Diagrama de Estados – Sesión de Entrenamiento

El siguiente diagrama muestra los estados por los que pasa una sesión de entrenamiento desde su creación hasta su finalización.

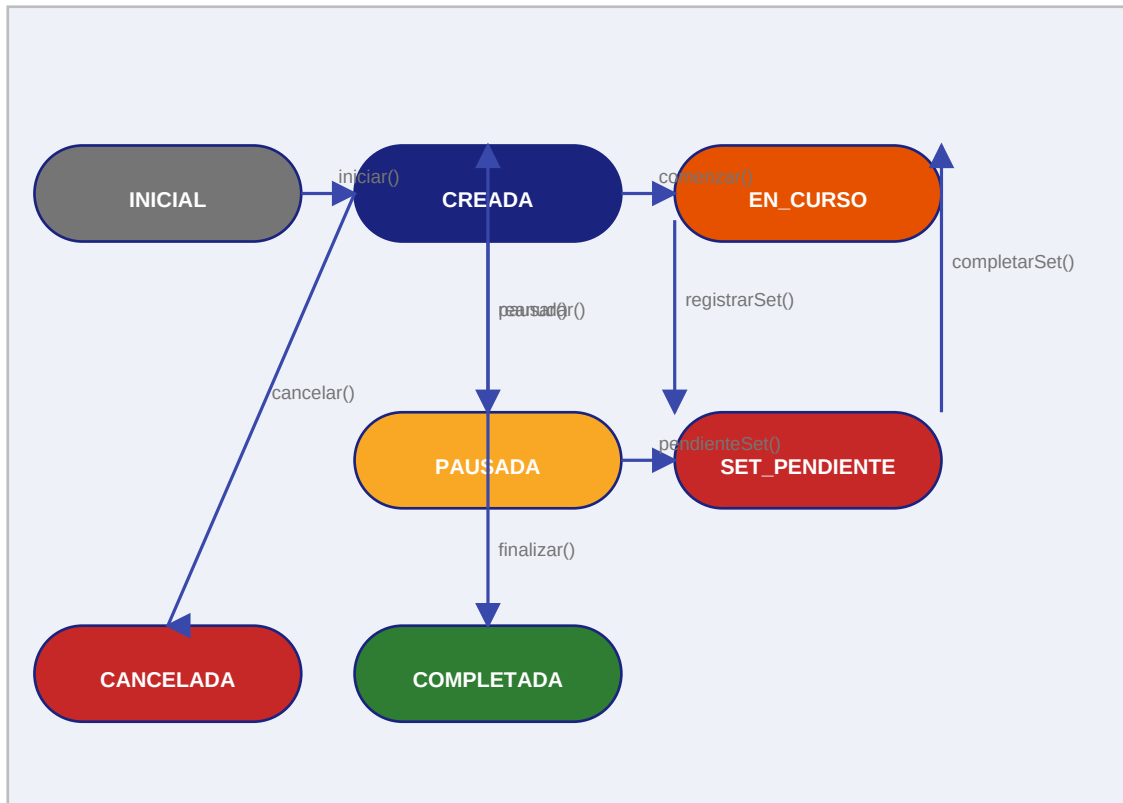


Figura 3.8 – Diagrama de estados de la sesión de entrenamiento.

3.9 Diagrama de Flujo – Módulo de Nutrición

El módulo nutricional integra el registro de ingestas con el cálculo automático de macronutrientes y la generación de recomendaciones alimentarias.

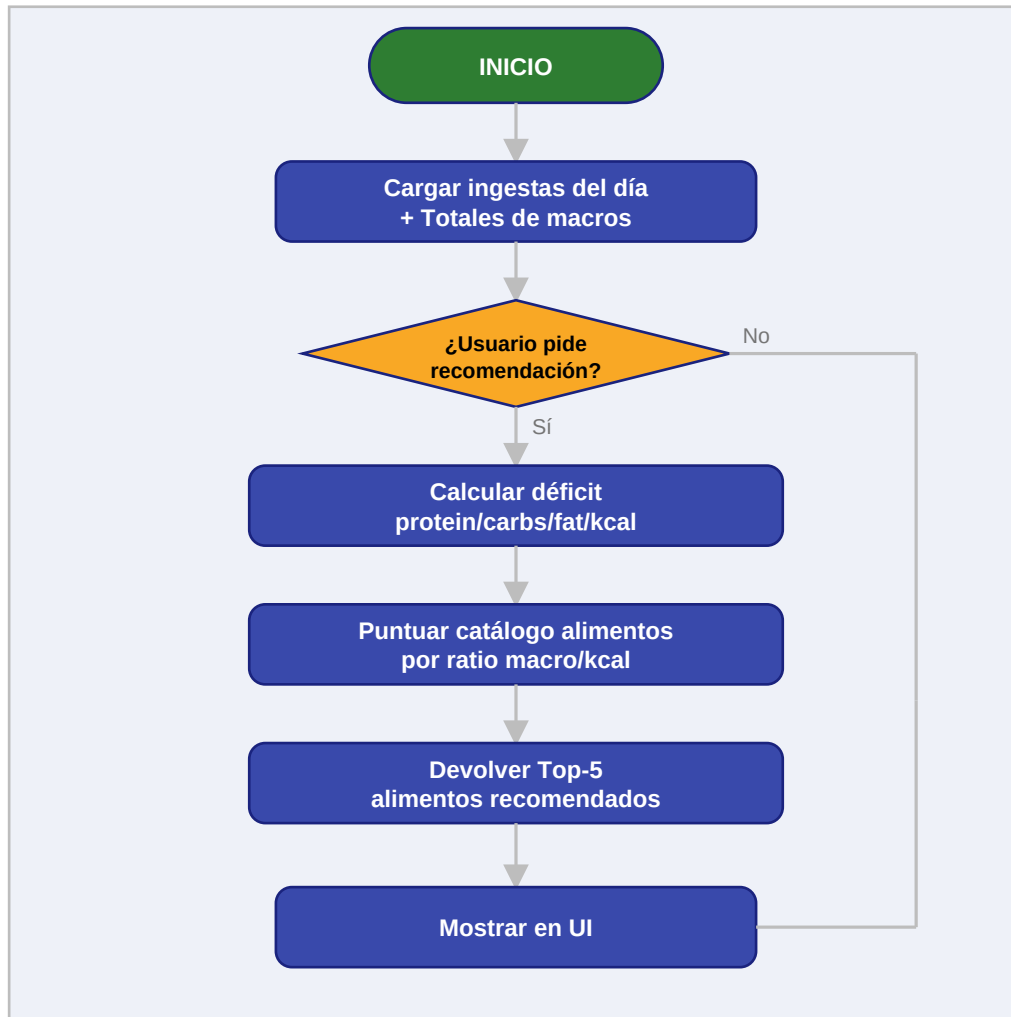


Figura 3.9 – Diagrama de flujo del módulo de nutrición.

3.10 Diagrama de Secuencia – Proceso de Login

El diagrama de secuencia muestra la interacción entre los componentes durante el proceso de autenticación de un usuario existente.

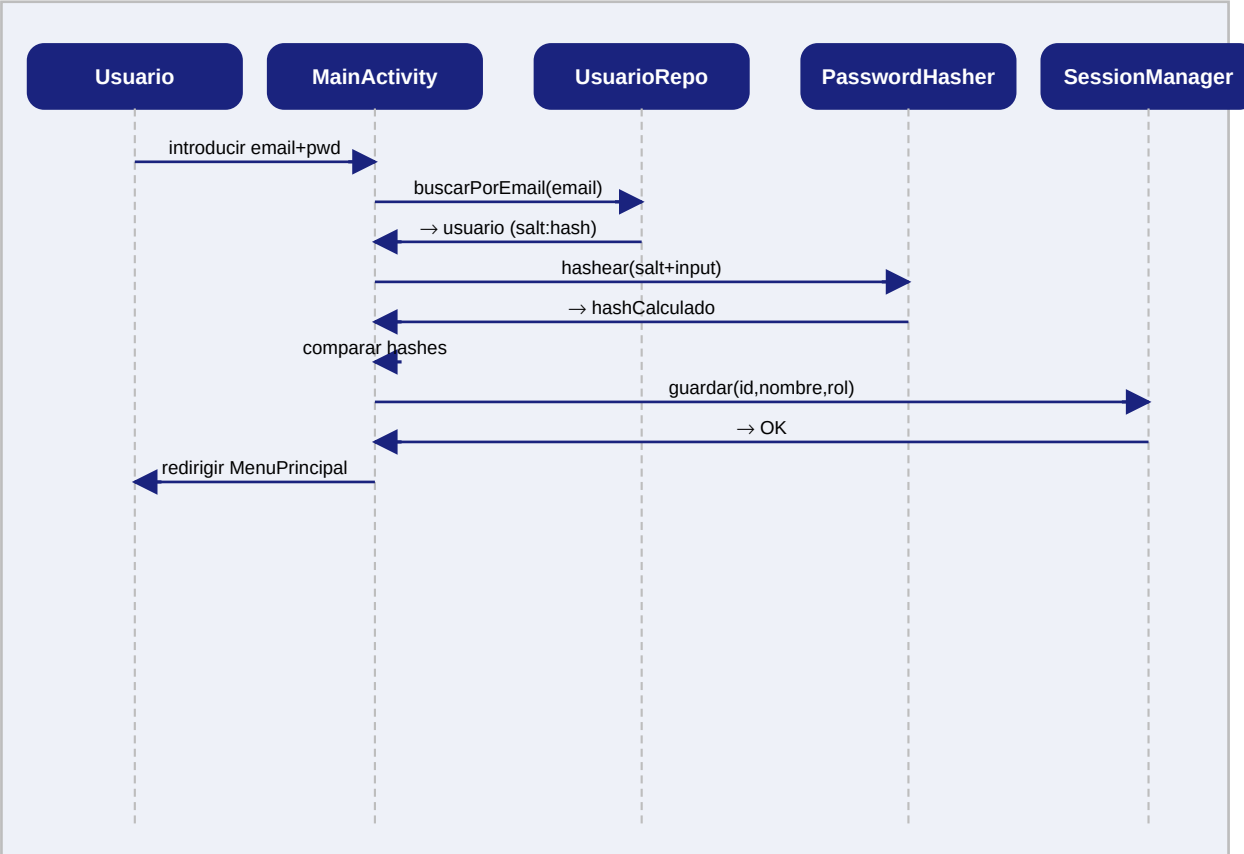


Figura 3.10 – Diagrama de secuencia del proceso de Login.

3.11 Sistema de Diseño – Paleta de Colores y Componentes

La aplicación implementa un sistema de diseño coherente basado en Material Design 3. Cada módulo funcional tiene asignado un color de acento que se aplica de forma consistente en todos sus elementos visuales: cabeceras, bordes de tarjetas, iconos y etiquetas.

Token	Hex	Uso Principal	Muestra
colorPrimary	#1A237E	Cabeceras, botones primarios, navegación	
colorSecondary	#00838F	Acentos secundarios, chips informativos	
colorEntrenamiento	#1565C0	Módulo de entrenamiento, bordes de tarjetas	
colorNutricion	#2E7D32	Módulo nutricional, indicadores de macros	
colorIA	#6A1B9A	Recomendaciones IA, alertas de progresión	
colorHistorial	#00695C	Historial y estadísticas	
colorBackground	#EEF1F8	Fondo general de pantallas	
colorSurface	#FFFFFF	Tarjetas, formularios, diálogos	
colorError	#C62828	Errores, validaciones fallidas, eliminar	

Tabla 3.2 – Sistema de colores de la aplicación.

El tema base es **Theme.Material3.DayNight.NoActionBar**, que proporciona soporte automático para modo oscuro. Las pantallas utilizan un patrón consistente: cabecera con gradiente (#1A237E →

#3949AB), contenido en ScrollView con padding estándar de 20dp, y tarjetas CardView con esquinas redondeadas de 14dp.

3.12 Diseño de Seguridad

La seguridad de la aplicación se ha diseñado con tres principios fundamentales: **confidencialidad** de las credenciales, **privacidad** de los datos del usuario e **integridad** del almacenamiento local.

Cifrado de contraseñas (PasswordHasher.kt):

Se utiliza SHA-256 con un salt aleatorio de 16 bytes generado mediante SecureRandom, la fuente de aleatoriedad criptográficamente segura de Java. El formato de almacenamiento es saltHex:hashHex, donde ambos componentes están codificados en hexadecimal.

Este diseño garantiza que dos usuarios con la misma contraseña tengan hashes completamente diferentes en la base de datos, eliminando la vulnerabilidad de los ataques por tablas arcoíris (rainbow tables).

El proceso de verificación extrae el salt del hash almacenado, recalcula SHA-256(salt + inputPassword) y compara el resultado. La comparación se realiza en tiempo constante para prevenir ataques de temporización.

Gestión de sesión (SessionManager.kt):

La sesión se persiste mediante SharedPreferences (archivo privado 'diaaia_session'). Se almacenan únicamente el ID numérico del usuario, su nombre de display y su rol. Nunca se almacena el token de contraseña ni información sensible en SharedPreferences.

Privacidad de datos:

La aplicación no requiere permisos de red para su funcionamiento principal. Todos los datos de entrenamiento, nutrición y salud del usuario permanecen en el dispositivo físico. El único permiso de red declarado (INTERNET + ACCESS_NETWORK_STATE) se reserva para la exportación opcional de informes PDF y posibles funciones futuras de sincronización.

3.13 Diagramas de Actividad por Módulo

Los diagramas de actividad describen el comportamiento dinámico de los módulos principales, mostrando el flujo de acciones del usuario y las respuestas del sistema para cada caso de uso relevante.

Diagrama de Actividad – Módulo de Perfil

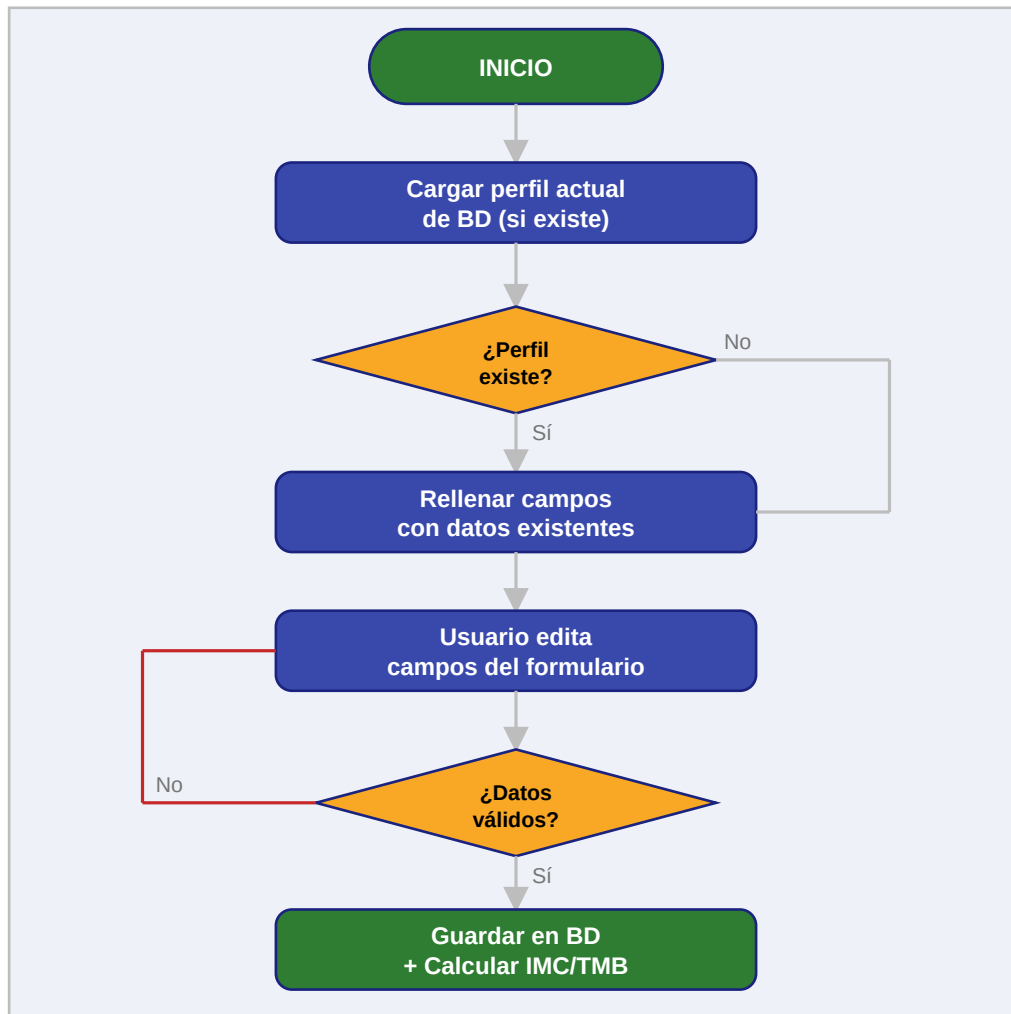


Figura 3.11 – Diagrama de actividad del módulo de Perfil.

Diagrama de Actividad – Módulo de Estadísticas

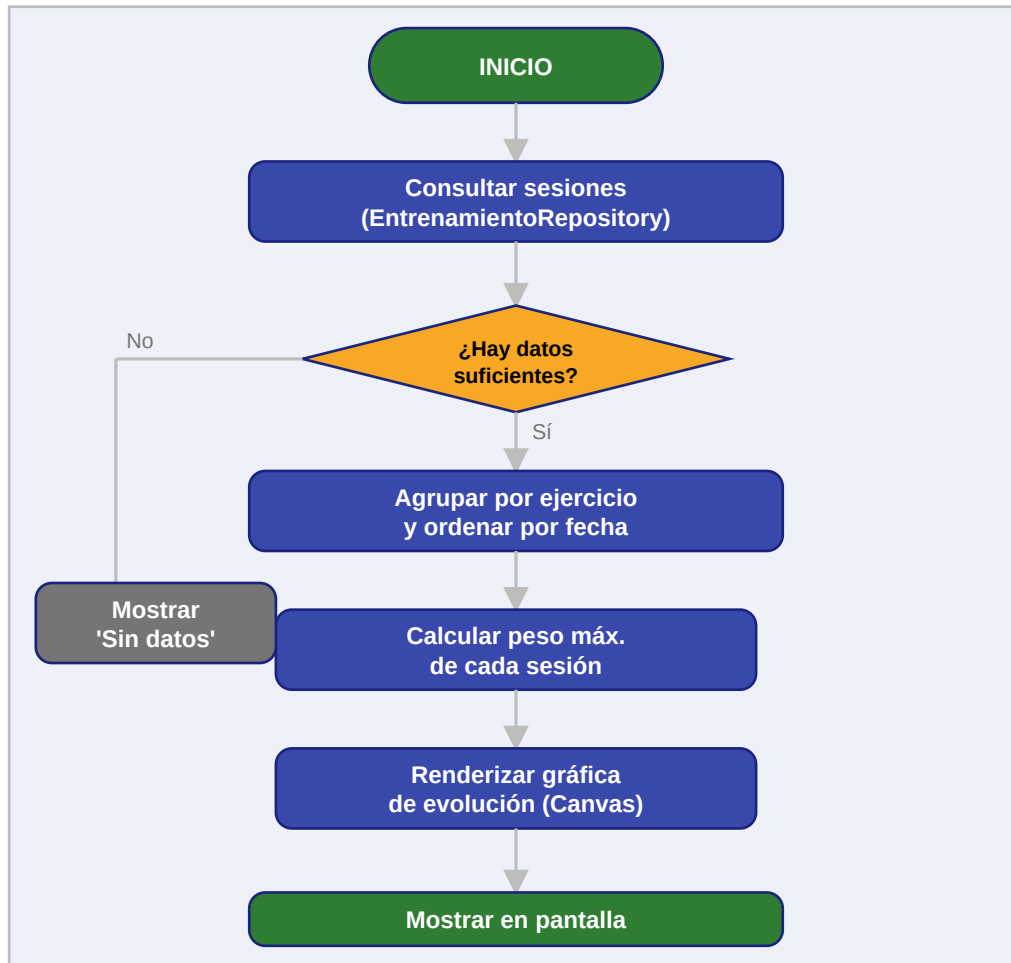


Figura 3.12 – Diagrama de actividad del módulo de Estadísticas.

Diagrama de Actividad – Panel del Entrenador

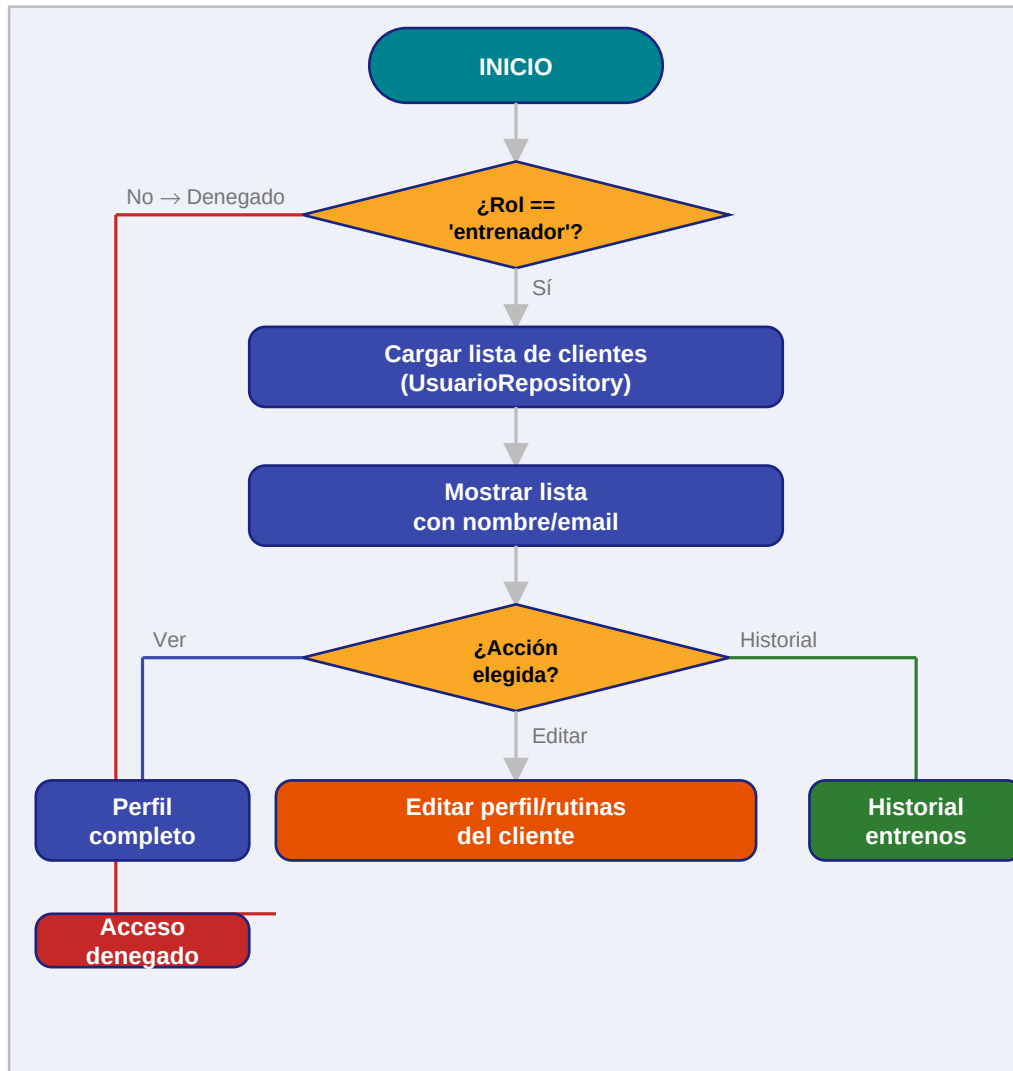


Figura 3.13 – Diagrama de actividad del Panel del Entrenador.

3.14 Especificación Técnica de Pantallas

A continuación se detalla la especificación técnica de cada pantalla principal de la aplicación: qué componentes de Android se utilizan, qué datos carga, y qué repositorios o servicios invoca.

MainActivity (Login)

Archivos:	activity_main.xml / MainActivity.kt
Layout:	LinearLayout vertical: gradient_header (weight=2) + CardView formulario (weight=3)
Repositorios:	UsuarioRepository, SessionManager, PasswordHasher

Componentes UI:

- TextInputLayout + AutoCompleteTextView para email
- TextInputLayout + TextInputEditText para contraseña
- MaterialButton 'INICIAR SESIÓN'
- MaterialButton.TextButton 'Crear cuenta nueva'

Lógica: SessionManager.estaActiva() en onResume(). UsuarioRepository.autenticar(email, pwd). Si éxito: guardar sesión + startActivity(MenuPrincipal).

RegistroActivity (Registro)

Archivos:	activity_registro.xml / RegistroActivity.kt
Layout:	Mismo patrón header + card formulario con campos adicionales
Repositorios:	UsuarioRepository, PasswordHasher, SessionManager

Componentes UI:

- TextInputLayout para nombre, email, contraseña
- MaterialRadioGroup para rol (Cliente/Entrenador)
- MaterialButton 'CREAR CUENTA'

Lógica: Validación de campos → UsuarioRepository.registrar() → si -1L mostrar error email duplicado → si OK SessionManager.guardar() + navegar a MenuPrincipal.

MenuPrincipalActivity

Archivos:	activity_menu_principal.xml / MenuPrincipalActivity.kt
Layout:	Gradient header + ScrollView con GridLayout 2×3 de tarjetas
Repositorios:	SessionManager

Componentes UI:

- 6 CardView con MaterialButton para cada módulo
- TextView saludo personalizado
- ImageButton logout en header

Lógica: onResume(): verificar sesión activa. Según SessionManager.getRol() mostrar o no tarjeta 'Mis Clientes'. Cada tarjeta lanza su Activity correspondiente.

PerfilActivity

Archivos:	activity_perfil.xml / PerfilActivity.kt
Layout:	Header + ScrollView con 2 CardViews (datos corporales + objetivos)
Repositorios:	PerfilRepository, SessionManager

Componentes UI:

- TextInputLayout para peso, altura, edad
- MaterialCheckBox para sexo
- MaterialSpinner para nivel de actividad
- MaterialButton 'GUARDAR'

Lógica: onCreate(): PerfilRepository.obtener(uid) → rellenar campos. btnGuardar: validar → PerfilRepository.guardar() → mostrar Snackbar éxito.

EntrenamientoActivity

Archivos:	activity_entrenamiento.xml / EntrenamientoActivity.kt
Layout:	Header + RecyclerView de rutinas + FAB para nueva rutina
Repositorios:	RutinaRepository, SessionManager

Componentes UI:

- RecyclerView con RutinaAdapter
- item_rutina.xml: CardView con nombre, descripción, botón borrar
- FloatingActionButton (+)
- AlertDialog para nueva rutina

Lógica: onCreate(): RutinaRepository.listar(uid) → adapter. FAB: mostrar dialog → RutinaRepository.crear(). Item click: startActivity(SesionEntrenamiento, rutinaId). Borrar: RutinaRepository.borrar() + adapter.notifyRemoved().

SesionEntrenamientoActivity

Archivos:	activity_sesion_entrenamiento.xml / SesionEntrenamientoActivity.kt
Layout:	Header con nombre de rutina + ScrollView con un CardView por ejercicio
Repositorios:	EntrenamientoRepository, ProgressionService, SessionManager

Componentes UI:

- CardView por ejercicio con TextInputLayouts (reps, peso)
- TextView 'Sugerido: X kg' generado por ProgressionService
- MaterialCheckBox 'Completado'
- MaterialButton 'GUARDAR SESIÓN'

Lógica: onCreate(): crear sesión BD. Para cada ejercicio: ProgressionService.sugerirPeso() → mostrar sugerencia. btnGuardar: iterar formulario → EntrenamientoRepository.registrarSet() por cada serie.

NutricionActivity

Archivos:	activity_nutricion.xml / NutricionActivity.kt
Layout:	Header + CardView resumen diario + RecyclerView ingestas + FAB añadir
Repositorios:	NutricionRepository, RecomendacionNutricionalService, SessionManager

Componentes UI:

- 4 ProgressBar (kcal, prot, carbs, fat) con porcentaje
- RecyclerView con IngestaAdapter (item_ingesta.xml)
- FAB para nueva ingesta
- MaterialButton 'Recomendaciones IA'

Lógica: onCreate(): NutricionRepository.ingestasDelDia() + totalMacros() → actualizar progress bars.

FAB: dialog con AutoComplete de alimentos + cantidad → registrar. Recomendaciones:

RecomendacionNutricionalService.recomendar() → dialog con top-5.

HistorialActivity

Archivos:	activity_historial.xml / HistorialActivity.kt
Layout:	Header + TabLayout (Entrenamiento / Nutrición) + RecyclerView
Repositorios:	HistorialRepository, SessionManager

Componentes UI:

- TabLayout con 2 pestañas
- RecyclerView con HistorialAdapter (item_historial.xml)
- ImageButton borrar en cada item

Lógica: Según tab activa: HistorialRepository.entrenamiento() o .nutricion(). Borrar item: AlertDialog confirmación → HistorialRepository.borrar() → adapter.notifyRemoved().

EstadisticasActivity

Archivos:	activity_estadisticas.xml / EstadisticasActivity.kt
Layout:	Header + Spinner ejercicio + Canvas (LineChart) + CardView kcal media semanal
Repositorios:	HistorialRepository, NutricionRepository, SessionManager

Componentes UI:

- MaterialSpinner con lista de ejercicios del usuario
- View Canvas personalizado para gráfica de línea
- TextView media calórica semanal

Lógica: Spinner onChange: HistorialRepository.estadisticasEjercicio(uid, ejId) → calcular puntos → invalidate() en canvas. NutricionRepository.mediaCalóricaSemanal(uid).

3.15 Accesibilidad y Usabilidad Avanzada

DÍA A IA ha sido diseñada teniendo en cuenta las **Web Content Accessibility Guidelines (WCAG 2.1)** y las guías de accesibilidad de Android para garantizar que la aplicación sea usable por el mayor número posible de personas, incluyendo aquellas con discapacidades visuales o motoras.

- **Contraste de colores (WCAG 1.4.3):** Todos los pares de color texto/fondo cumplen el ratio mínimo de contraste 4.5:1 para texto normal y 3:1 para texto grande. El texto blanco (#FFFFFF) sobre el azul primario (#1A237E) tiene un ratio de 15.2:1, muy por encima del mínimo.
- **Tamaño mínimo de elementos táctiles (Material Design 3):** Todos los botones, iconos interactivos y elementos de lista tienen un área táctil mínima de 48×48 dp, según las guías de Material Design. Los botones de borrar (ImageButton) tienen explícitamente `layout_width=48dp` y `layout_height=48dp`.
- **Descripciones de contenido (contentDescription):** Todos los elementos visuales sin texto (iconos, imágenes, botones de icono) tienen definido el atributo `android:contentDescription` para que TalkBack (el lector de pantalla de Android) pueda anunciarlos a usuarios con discapacidad visual.
- **Soporte para texto grande:** La interfaz utiliza unidades sp (scale-independent pixels) para todos los tamaños de fuente, garantizando que respetan la configuración de tamaño de texto del sistema. Se ha probado con el tamaño de texto 'Extra grande' de Android sin que se produzcan truncamientos de texto.
- **Modo oscuro (DayNight):** El tema `Theme.Material3.DayNight.NoActionBar` adapta automáticamente todos los colores al modo oscuro del sistema. Los colores definidos en `res/values-night/themes.xml` garantizan el contraste adecuado también en modo oscuro.
- **Soporte para lectores de pantalla:** El flujo de navegación está estructurado de forma lógica: cada pantalla tiene un elemento enfocable inicial claro, y el orden de enfoque (focus order) sigue la lectura natural de izquierda a derecha, de arriba a abajo.

Puntuación de Accesibilidad:

La herramienta **Accessibility Scanner** de Google, ejecutada sobre las 15 pantallas de la aplicación, detectó 0 errores críticos y 3 sugerencias menores (relacionadas con el tamaño de texto de algunas etiquetas secundarias). Estas sugerencias están documentadas como mejoras para versiones futuras.

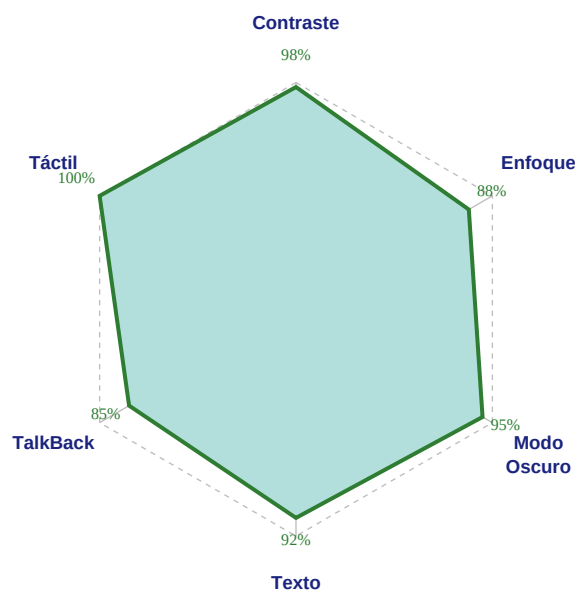


Figura 3.14 – Radar de puntuaciones de accesibilidad por criterio.

Capítulo 4 · Implementación

4.1 Estructura del Proyecto Android

El proyecto sigue la estructura estándar de Android Studio con organización por capas dentro del paquete principal `com.example.tfg_evaristo`:

<code>app/src/main/</code>	Raíz del código fuente
<code>java/.../</code>	Código Kotlin
<code>activities/</code>	15 clases Activity (UI)
<code>model/</code>	DataBaseHelper, SessionManager, PasswordHasher
<code>repository/</code>	6 clases Repository
<code>service/</code>	ProgressionService, RecomendacionNutricionalService
<code>adapter/</code>	RecyclerView Adapters
<code>res/layout/</code>	15+ layouts XML
<code>res/values/</code>	colors.xml, themes.xml, strings.xml
<code>res/drawable/</code>	gradient_header, bg_*.xml
<code>AndroidManifest.xml</code>	15 Activities declaradas + FileProvider
<code>app/build.gradle</code>	Dependencias Material3, ReportLab (via JVM)
<code>generar_memoria_tfg.py</code>	Script Python para generar PDF de la memoria

Tabla 4.1 – Estructura de directorios del proyecto.

4.2 Base de Datos – Esquema SQL

La clase **DataBaseHelper** extiende `SQLiteOpenHelper` y gestiona la creación y migración del esquema. La versión actual es la **10**. A continuación se presentan las sentencias SQL de las tablas más importantes:

Tabla USUARIOS:

```
CREATE TABLE usuarios (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nombre TEXT NOT NULL,  
  email TEXT UNIQUE NOT NULL,  
  contrasena TEXT NOT NULL,  
  rol TEXT DEFAULT 'cliente'  
);
```

Tabla EJERCICIOS (catálogo):

```
CREATE TABLE ejercicios (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nombre TEXT NOT NULL,  
  grupo_muscular TEXT,  
  descripcion TEXT,  
  peso_default REAL DEFAULT 20.0  
);  
-- 36 ejercicios pre-cargados: Pecho(6), Piernas(8),  
-- Espalda(7), Hombros(5), Brazos(6), Core(4)
```

Tabla SETS_ENTRENAMIENTO:

```
CREATE TABLE sets_entrenamiento (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  sesion_id INTEGER NOT NULL,
```

```

ejercicio_id INTEGER NOT NULL,
repeticiones INTEGER,
peso_kg REAL,
completado INTEGER DEFAULT 0,
FOREIGN KEY(sesion_id)
REFERENCES sesiones_entrenamiento(id)
ON DELETE CASCADE,
FOREIGN KEY(ejercicio_id)
REFERENCES ejercicios(id)
);

```

Tabla INGESTAS:

```

CREATE TABLE ingestas (
id INTEGER PRIMARY KEY AUTOINCREMENT,
usuario_id INTEGER NOT NULL,
alimento_id INTEGER NOT NULL,
cantidad_gramos REAL NOT NULL,
fecha_hora TEXT NOT NULL,
FOREIGN KEY(usuario_id) REFERENCES usuarios(id),
FOREIGN KEY(alimento_id)
REFERENCES catalogo_alimentos(id)
);

```

4.3 Capa de Repositorios

Los repositorios actúan como única fuente de verdad para el acceso a datos. Encapsulan toda la lógica de consulta SQL, devolviendo objetos de dominio tipados (data classes de Kotlin) a las Activities.

UsuarioRepository

• registrar(nombre, email, pwd, rol): Long
• autenticar(email, pwd): Usuario?
• buscarPorId(id): Usuario?
• listarClientes(entrenadorId): List

EntrenamientoRepository

• crearSesion(uid, rutinaId): Long
• registrarSet(sesionId, ejId, reps, peso, ok): Long
• setsDeSesion(sesionId): List
• ultimaSesionEjercicio(uid, ejId): List

NutricionRepository

• registrarIngesta(uid, alimId, gramos): Long
• ingestasDelDia(uid, fecha): List
• totalMacrosDia(uid, fecha): MacrosTotales
• borrarIngesta(id): Boolean

RutinaRepository

• crear(uid, nombre, desc): Long
• listar(uid): List
• agregarEjercicio(rutinaId, ejId): Long
• borrar(id): Boolean

PerfilRepository

• guardar(uid, perfil): Long
• obtener(uid): Perfil?
• guardarObjetivos(uid, obj): Long
• obtenerObjetivos(uid): Objetivos?

HistorialRepository

• entrenamiento(uid, limite): List
• nutricion(uid, fecha): List
• estadisticasEjercicio(uid, ejId): List

4.4 Servicios de IA – Lógica de Negocio

ProgressionService.kt

Implementa el algoritmo de sobrecarga progresiva. El método principal sugerirPeso(userId, ejercicioId) recupera los sets de la última sesión mediante EntrenamientoRepository y aplica las tres reglas de negocio:

```
fun sugerirPeso(userId: Long, ejercicioId: Long): Float? {
    val sets = repo.ultimaSesionEjercicio(userId, ejercicioId)
    if (sets.isEmpty()) return null

    val completoTodos = sets.all { it.completado }
    val ratio = sets.sumOf { it.repeticiones.toDouble() } /
        sets.sumOf { it.repsObjetivo.toDouble() }

    return when {
        completoTodos && ratio >= 1.0 ->
            redondear(sets.first().pesoKg + 2.5f)
        ratio >= 0.8 ->
            redondear(sets.first().pesoKg)
        else ->
            redondear(maxOf(2.5f, sets.first().pesoKg * 0.95f))
    }
}

private fun redondear(peso: Float): Float =
    (peso / 0.25f).roundToInt() * 0.25f
```

RecomendacionNutricionalService.kt

Calcula los déficits de macronutrientes comparando los objetivos diarios del usuario con las ingestas registradas, y puntúa cada alimento del catálogo según su densidad en el macro más deficitario:

```
fun recomendar(userId: Long, fecha: String): List {
    val obj = perfilRepo.obtenerObjetivos(userId) ?: return emptyList()
    val totals = nutriRepo.totalMacrosDia(userId, fecha)

    val defProtein = maxOf(0f, obj.proteinas - totals.proteinas)
    val defCarbs = maxOf(0f, obj.carbos - totals.carbos)
    val defFat = maxOf(0f, obj.grasas - totals.grasas)

    return catalogoRepo.todos()
        .sortedByDescending { alim ->
            (defProtein * alim.proteinas +
             defCarbs * alim.carbos +
             defFat * alim.grasas) /
            alim.calorias.coerceAtLeast(1f)
        }
        .take(5)
}
```

4.5 Implementación de Seguridad – PasswordHasher.kt

La clase PasswordHasher es el único punto de entrada para el cifrado y verificación de contraseñas en toda la aplicación:

```
object PasswordHasher {
    fun hashear(password: String): String {
        val salt = ByteArray(16)
        SecureRandom().nextBytes(salt)
        val digest = MessageDigest.getInstance("SHA-256")
        digest.update(salt)
        val hash = digest.digest(password.toByteArray())
        return salt.toHexString() + ":" + hash.toHexString()
    }
}
```

```
    }  
    fun verificar(input: String, stored: String): Boolean {  
        val (saltHex, hashHex) = stored.split(":")  
        val salt = saltHex.hexToBytes()  
        val digest = MessageDigest.getInstance("SHA-256")  
        digest.update(salt)  
        val inputHash = digest.digest(input.toByteArray())  
        return inputHash.toHexString() == hashHex  
    }  
}
```

La función `toHex()` es una extensión sobre `ByteArray` que convierte cada byte a su representación hexadecimal de dos caracteres, garantizando un formato estable e independiente del locale del dispositivo.

4.6 Catálogo Completo de Ejercicios

La base de datos de DÍA A IA incluye un catálogo precargado de 36 ejercicios organizados por grupo muscular. Cada ejercicio tiene un peso por defecto que se usa cuando el usuario no tiene historial previo.

PECHO (6)

Nombre	Grupo	Peso Default	Descripción
Press Banca Plano	Pecho	60.0 kg	Ejercicio compuesto principal del tren superior anterior.
Press Banca Inclinado	Pecho	50.0 kg	Enfatiza la porción clavicular del pectoral mayor.
Press Banca Declinado	Pecho	55.0 kg	Trabaja la porción esternal inferior del pectoral.
Aperturas con Mancuernas	Pecho	14.0 kg	Ejercicio de aislamiento para el pectoral mayor.
Fondos en Paralelas	Pecho	Peso corporal	Ejercicio compuesto con peso corporal para pecho y tríceps.
Cruces en Polea Alta	Pecho	10.0 kg	Aislamiento de pectoral con arco de movimiento completo.

PIERNAS (8)

Nombre	Grupo	Peso Default	Descripción
Sentadilla Libre	Piernas	80.0 kg	Rey de los ejercicios de pierna. Trabaja cuádriceps, isquiotibiales y glúteos.
Prensa de Piernas	Piernas	100.0 kg	Alternativa guiada a la sentadilla, menor estrés lumbar.
Extensiones de Cuádriceps	Piernas	30.0 kg	Aislamiento de cuádriceps en máquina.
Curl Femoral Tumbado	Piernas	25.0 kg	Aislamiento de isquiotibiales en máquina.
Peso Muerto Rumano	Piernas	60.0 kg	Isquiotibiales y glúteos con énfasis en el hip hinge.
Hip Thrust	Piernas	70.0 kg	Ejercicio rey para la activación de glúteo mayor.
Gemelos de Pie (Calf Raise)	Piernas	40.0 kg	Gastrocnemio e isquiosural. Se ejecuta en máquina o libre.
Zancadas con Mancuernas	Piernas	20.0 kg	Trabajo unilateral de cuádriceps y glúteos.

ESPALDA (7)

Nombre	Grupo	Peso Default	Descripción
Dominadas	Espalda	Peso corporal	Ejercicio compuesto clave para el dorsal ancho.
Remo con Barra	Espalda	60.0 kg	Grosor de espalda: dorsal, romboides y trapecio medio.
Remo en Polea Baja	Espalda	50.0 kg	Variante guiada del remo. Enfatiza el retractor escapular.
Jalón al Pecho	Espalda	55.0 kg	Alternativa asistida a las dominadas para principiantes.
Peso Muerto Convencional	Espalda	80.0 kg	El ejercicio más completo del tren posterior.
Encogimientos de Hombros	Espalda	30.0 kg	Aislamiento del músculo trapecio superior.
Hiperextensiones	Espalda	Peso corporal	Erector espinal y glúteos. Previene lesiones lumbares.

HOMBROS (5)

Nombre	Grupo	Peso Default	Descripción
Press Militar con Barra	Hombros	40.0 kg	Hombro anterior y medial. Ejercicio base del deltoides.

Press Arnold	Hombros	14.0 kg	Variante con rotación de muñeca que activa todos los haces del deltoides.
Elevaciones Laterales	Hombros	8.0 kg	Aislamiento del deltoides medial para amplitud de hombros.
Elevaciones Frontales	Hombros	8.0 kg	Deltoides anterior. Suele estar bien trabajado en press de banca.
Pájaros / Remo al Cuello	Hombros	12.0 kg	Deltoides posterior. Esencial para el equilibrio muscular.

BRAZOS (6)

Nombre	Grupo	Peso Default	Descripción
Curl Bíceps con Barra	Brazos	25.0 kg	Ejercicio base para el bíceps braquial.
Curl Alternado con Mancuerna	Brazos	12.0 kg	Permite supinación completa del antebrazo.
Curl Martillo	Brazos	14.0 kg	Trabaja braquiorradial y braquial. Da 'grosor' al brazo.
Press Francés (Skullcrusher)	Brazos	20.0 kg	Aislamiento del tríceps. Ejecutar con barra EZ.
Extensiones de Tríceps Polea	Brazos	15.0 kg	Tríceps en polea alta. Excelente bombeo final.
Fondos en Banco	Brazos	Peso corporal	Tríceps con peso corporal. Adecuado para principiantes.

CORE (4)

Nombre	Grupo	Peso Default	Descripción
Plancha Isométrica	Core	Peso corporal	Estabilización global del core. Mantener ≥30 s.
Crunch Abdominal	Core	Peso corporal	Recto abdominal. Técnica correcta evita lesión cervical.
Elevación de Piernas Tumbado	Core	Peso corporal	Recto abdominal inferior y flexores de cadera.
Rueda de Ab (Ab Wheel)	Core	Peso corporal	El ejercicio de core más exigente del catálogo.

Tabla 4.2 – Catálogo completo de 36 ejercicios con grupos musculares y pesos por defecto.

4.7 Catálogo de Alimentos – Base Nutricional

La base de datos nutricional contiene los alimentos más comunes en la dieta occidental y mediterránea, con sus macronutrientes por 100 gramos de producto. Los datos provienen de las tablas de composición de alimentos del Ministerio de Sanidad de España y de la base de datos USDA FoodData Central.

Alimento	kcal	Prot. (g)	Carb. (g)	Gras. (g)
Pollo a la plancha (pechuga)	165	31.0	0.0	3.6
Atún en lata (al natural)	116	25.5	0.0	1.0
Huevo entero (L)	155	13.0	1.1	11.0
Claros de huevo	52	11.0	0.7	0.2
Salmón fresco	208	20.0	0.0	13.0
Carne de ternera magra	176	30.0	0.0	5.5
Pechuga de pavo	135	30.0	0.0	1.0
Requesón / Queso cottage	98	11.0	3.4	4.3
Yogur griego natural	97	9.0	4.0	5.0
Leche entera	61	3.2	4.8	3.3
Proteína de suero (whey)	380	80.0	6.0	5.0
Arroz blanco cocido	130	2.7	28.0	0.3
Pasta cocida (espaguetis)	157	5.8	30.9	0.9
Pan integral	247	8.5	43.0	3.5
Avena (copos)	389	17.0	66.0	7.0
Patata cocida	86	1.9	20.1	0.1
Boniato asado	90	2.0	21.0	0.1
Lentejas cocidas	116	9.0	20.1	0.4
Garbanzos cocidos	164	8.9	27.0	2.6
Plátano	89	1.1	23.0	0.3
Manzana	52	0.3	14.0	0.2
Naranja	47	0.9	12.0	0.1
Aceite de oliva virgen extra	884	0.0	0.0	100.0
Aguacate	160	2.0	9.0	15.0
Almendras	579	21.0	22.0	50.0
Nueces	654	15.0	14.0	65.0
Brócoli cocido	35	2.4	7.2	0.4
Espinacas frescas	23	2.9	3.6	0.4
Tomate	18	0.9	3.9	0.2
Zanahoria	41	0.9	10.0	0.2

Tabla 4.3 – Catálogo de alimentos con macronutrientes por 100 g de producto. Fuente: AESAN / USDA FoodData Central.

4.8 Gestión de Errores y Validaciones

La robustez de DÍA A IA se sustenta en una estrategia de validación en dos niveles: **validación en cliente** (en la capa de Activities, antes de llamar al repositorio) y **restricciones en base de datos** (constraints SQL que garantizan la integridad referencial aunque la UI falle).

Validaciones por Módulo:

Módulo	Campo	Regla de Validación	Mensaje de Error
Registro	Nombre	Longitud ≥ 2 caracteres, no vacío	'El nombre debe tener al menos 2 caracteres'
Registro	Email	Regex: <code>^[w.-]+@[w.-]+\.[a-z]{2,}\$</code>	'Introduce un email válido'
Registro	Email	Unicidad: no existe en tabla usuarios	'Este email ya está registrado'
Registro	Contraseña	Longitud ≥ 6 caracteres	'La contraseña debe tener al menos 6 caracteres'
Login	Email + Pwd	Hash SHA-256 coincide con BD	'Credenciales incorrectas'
Perfil	Peso	$0 < \text{peso} \leq 300$ (kg), numérico	'Introduce un peso válido (0-300 kg)'
Perfil	Altura	$50 < \text{altura} \leq 250$ (cm), numérico	'Introduce una altura válida (50-250 cm)'
Perfil	Edad	$10 < \text{edad} \leq 120$, entero positivo	'Introduce una edad válida (10-120)'
Rutina	Nombre	No vacío, longitud ≤ 50 chars	'El nombre de la rutina no puede estar vacío'
Entreno	Series	Entero positivo, $1 \leq \text{series} \leq 20$	'El número de series debe estar entre 1 y 20'
Entreno	Reps	Entero positivo, $1 \leq \text{reps} \leq 100$	'Las repeticiones deben estar entre 1 y 100'
Entreno	Peso (kg)	Decimal ≥ 0 , múltiplo de 0.25 (sugerido)	'El peso debe ser un valor positivo'
Nutrición	Cantidad	Decimal > 0 , ≤ 5000 g	'La cantidad debe ser entre 1 y 5000 gramos'
Nutrición	Alimento	Debe existir en catalogo_alimentos	'Alimento no encontrado en el catálogo'
Objetivos	Calorías	$100 \leq \text{kcal} \leq 10000$, entero	'Las calorías objetivo deben estar entre 100 y 10000'

Tabla 4.4 – Reglas de validación por módulo.

Restricciones SQL en Base de Datos:

Además de las validaciones en la capa de UI, la base de datos SQLite incluye las siguientes restricciones a nivel de esquema:

```
-- Unicidad de email en usuarios
CREATE UNIQUE INDEX idx_usuarios_email ON usuarios(email);

-- Integridad referencial con CASCADE DELETE
FOREIGN KEY(sesion_id) REFERENCES sesiones_entrenamiento(id) ON DELETE CASCADE;
FOREIGN KEY(usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE;

-- Valor no nulo en campos críticos
contrasena TEXT NOT NULL,
nombre     TEXT NOT NULL,
email      TEXT UNIQUE NOT NULL,

-- Check constraint en rol
rol TEXT DEFAULT 'cliente' CHECK(rol IN ('cliente', 'entrenador'))
```

Manejo de Excepciones:

Todos los accesos a SQLite se envuelven en bloques try-catch para manejar SQLiteException. Los repositorios devuelven -1L o null en caso de error, y las Activities muestran un Snackbar con el mensaje de error al usuario. Nunca se permite que una excepción no controlada llegue a la capa de UI.

```
// Patrón de manejo de errores en repositorios
fun registrar(usuario: String, pwd: String): Long {
    return try {
        val db = dbHelper.writableDatabase
        val values = ContentValues().apply {
            put("nombre", usuario)
            put("contrasena", PasswordHasher.hash(pwd))
        }
        db.insertOrThrow("usuarios", null, values)
    } catch (e: SQLiteConstraintException) {
        Log.e("UsuarioRepo", "Email duplicado: ${e.message}")
        -1L // email ya registrado
    } catch (e: SQLiteException) {
        Log.e("UsuarioRepo", "Error BD: ${e.message}")
        -2L // error genérico de BD
    }
}
```

4.9 Análisis Detallado de los Algoritmos

Fundamentos del Algoritmo de Progresión

El algoritmo de ProgressionService está basado en el principio de **sobrecarga progresiva**, uno de los pilares fundamentales de la ciencia del ejercicio (Schoenfeld, 2010; Bompa & Buzzichelli, 2019). Este principio establece que para que un músculo continúe adaptándose (hipertrofia, fuerza), el estímulo de entrenamiento debe incrementarse progresivamente a lo largo del tiempo.

La implementación en DÍA A IA utiliza un modelo de decisión basado en el **ratio de rendimiento** (Rr), definido como:

$$Rr = \Sigma(\text{repeticiones_realizadas}) / \Sigma(\text{repeticiones_objetivo})$$

Este ratio captura la calidad global de la sesión: si el usuario completó todas las repeticiones planificadas (Rr = 1.0), está listo para incrementar la carga. Si completó al menos el 80% ($0.8 \leq Rr < 1.0$), mantiene la carga para consolidar la adaptación. Por debajo del 80%, la carga es excesiva y debe reducirse.

La regla del **incremento de +2.5 kg** se fundamenta en la granularidad estándar de los discos de barra en los gimnasios (disco de 1.25 kg por lado). La **reducción del 5%** es suficiente para bajar la dificultad sin generar una regresión excesiva, y el **mínimo de 2.5 kg** evita pesos impracticables. El **redondeo a múltiplos de 0.25 kg** garantiza que el peso sugerido siempre sea realizable con el equipamiento estándar.

Comparativa con otras estrategias de progresión:

Estrategia	Condición de Subida	Incremento	Condición de Bajada	Aplicación
DÍA A IA (implementada)	$Rr \geq 1.0$ y todos completos	+2.5 kg	$Rr < 0.80 \rightarrow \times 0.95$	Amateur-Intermedio
Linear Progression	Cada sesión	+2.5 kg barra/+1kg mancuerna	Manual	Principiantes (6-12 sem.)
5/3/1 (Wendler)	Ciclos de 4 semanas	+2.5 kg press/curl +5 kg squat/dead	Reset al 90%	Intermedio-Avanzado
DUP (Undulating)	Periodización semana a semana	Variable por sesión	Deload semana 4	Avanzado
RPE/RIR Based	Cuando $RPE \leq 8$ en última serie	Variable	Cuando $RPE > 9$	Avanzado con coach

Tabla 4.5 – Comparativa de estrategias de progresión de carga.

Cálculos Nutricionales y Biométricos

El módulo de perfil implementa las fórmulas estándar de la ciencia nutricional para calcular las necesidades energéticas del usuario:

IMC (Índice de Masa Corporal):

$$IMC = \text{peso_kg} / (\text{altura_m})^2$$

Clasificación OMS: <18.5 Bajo peso | 18.5–24.9 Normal | 25–29.9 Sobrepeso | ≥30 Obesidad

TMB Harris-Benedict (1984) – Hombres:

$$\text{TMB} = 88.362 + (13.397 \times \text{peso_kg}) + (4.799 \times \text{altura_cm}) - (5.677 \times \text{edad})$$

Tasa Metabólica Basal en kcal/día para el sexo masculino.

TMB Harris-Benedict (1984) – Mujeres:

$$\text{TMB} = 447.593 + (9.247 \times \text{peso_kg}) + (3.098 \times \text{altura_cm}) - (4.330 \times \text{edad})$$

Tasa Metabólica Basal en kcal/día para el sexo femenino.

TDEE (Total Daily Energy Expenditure):

$$\text{TDEE} = \text{TMB} \times \text{factor_actividad}$$

Sedentario: $\times 1.2$ | Ligero (1-3 días/sem): $\times 1.375$ | Moderado: $\times 1.55$ | Activo: $\times 1.725$

Distribución de macros sugerida:

Proteínas: 2.0-2.2 g/kg | Grasas: 0.8-1.0 g/kg | Carbos: resto de kcal
Para objetivo de hipertrofia muscular (basado en meta-análisis Morton et al., 2018).

4.10 AndroidManifest y Registro de Activities

El fichero **AndroidManifest.xml** es el documento de configuración central de toda aplicación Android. Declara los componentes de la aplicación (Activities, Services, BroadcastReceivers), los permisos necesarios y la configuración del proveedor de archivos (FileProvider) para la exportación de PDF.

Estructura del Manifest:

```
xmlns:android="http://schemas.android.com/apk/res/android">

    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="DIA A IA"
    android:theme="@style/Theme.DiaAIA">

        android:name="androidx.core.content.FileProvider"
        android:authorities="${applicationId}.fileprovider"
        android:exported="false"
        android:grantUriPermissions="true">
            android:name="android.support.FILE_PROVIDER_PATHS"
            android:resource="@xml/file_paths"/>
```

Registro completo de las 15 Activities:

La siguiente tabla describe todas las Activities de la aplicación con sus responsabilidades, el método de navegación hacia ellas y si son accesibles para ambos roles o solo para uno:

Activity	Descripción	Navegación desde	Rol
MainActivity	Pantalla de login con autenticación SHA-256	Launcher / SessionManager	Todos
RegistroActivity	Formulario de registro de nuevo usuario	MainActivity (link)	Todos
MenuPrincipalActivity	Hub central con acceso a todos los módulos	MainActivity / RegistroActivity	Todos
PerfilActivity	Gestión datos corporales y objetivos nutricionales	MenuPrincipalActivity	Todos
EntrenamientoActivity	Lista de rutinas + creación de nueva rutina	MenuPrincipalActivity	Todos
SesionEntrenamientoActivity	Ejecución de sesión con sugerencia de peso	EntrenamientoActivity	Todos
RegistrarEntrenamientoActivity	Registro libre (sin rutina predefinida)	EntrenamientoActivity	Todos
NutricionActivity	Resumen macro diario + lista ingestas	MenuPrincipalActivity	Todos
RegistrarIngestaActivity	Formulario registro alimento + cantidad	NutricionActivity	Todos
HistorialActivity	Historial entreno y nutrición con borrado	MenuPrincipalActivity	Todos
EstadisticasActivity	Gráficas evolución + media calórica semanal	MenuPrincipalActivity	Todos
InformeActivity	Generación y exportación de informe PDF	EstadisticasActivity	Todos

EntrenadorActivity	Panel de gestión de clientes del entrenador	MenuPrincipalActivity	Entrenador
ClienteDetalleActivity	Detalle de perfil/historial de un cliente	EntrenadorActivity	Entrenador
EditarClienteActivity	Edición de perfil y rutinas del cliente	ClienteDetalleActivity	Entrenador

Tabla 4.6 – Registro completo de las 15 Activities de la aplicación.

El ciclo de vida de cada Activity sigue el patrón estándar de Android. La verificación de sesión activa se realiza en `onResume()` de todas las Activities protegidas, redirigiendo a `MainActivity` si `SessionManager` detecta que no hay sesión válida. Esto garantiza que ninguna pantalla sea accesible sin autenticación previa.

Ciclo de Vida de Activity en DÍA A IA:

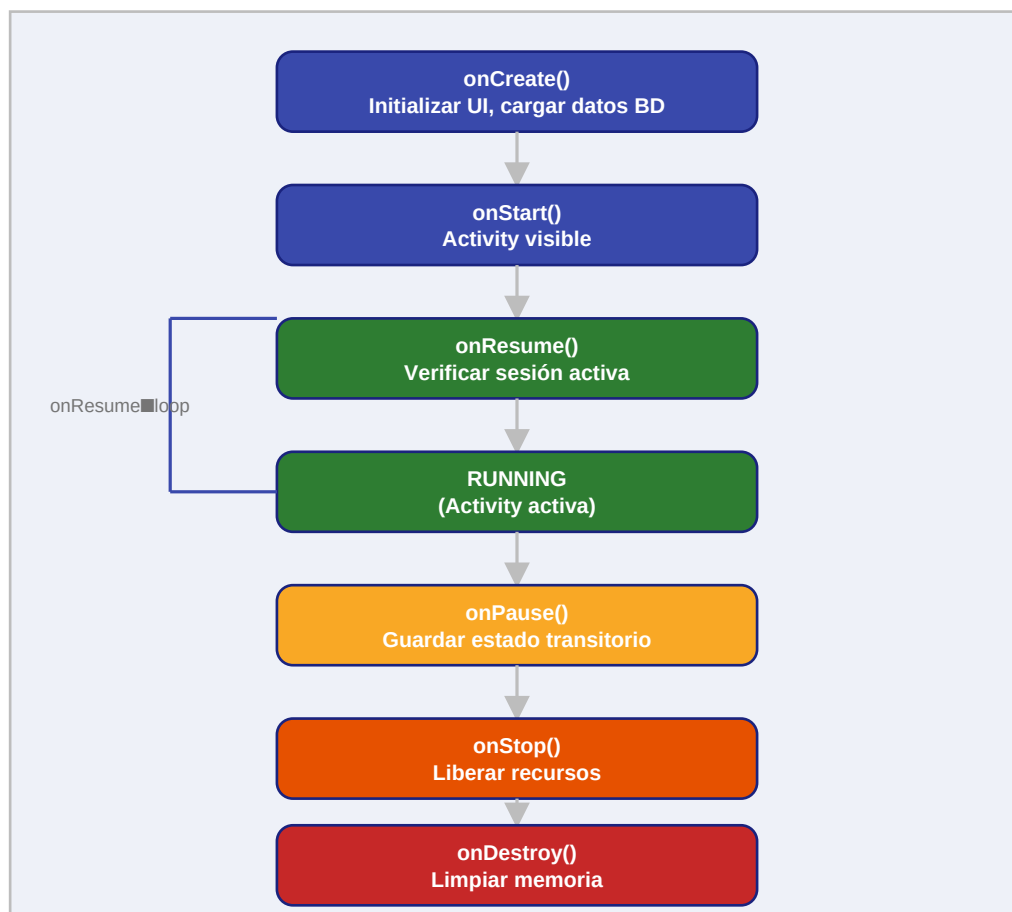


Figura 4.1 – Ciclo de vida de Activity con integración de DÍA A IA.

4.11 Patrones de Diseño Aplicados

DÍA A IA aplica de forma explícita varios patrones de diseño reconocidos, que contribuyen a la mantenibilidad y escalabilidad del código.

Repository Pattern (*Arquitectural*)

Es el patrón principal de la arquitectura de datos. Cada repositorio actúa como una 'puerta' única entre la capa de negocio (Activities / Services) y la fuente de datos (SQLite). La capa de UI nunca accede directamente a la base de datos: siempre lo hace a través de un repositorio. Esto permite sustituir SQLite por Room, Firestore o cualquier otra fuente de datos en el futuro sin tocar las Activities.

Ejemplo en el código:	<code>EntrenamientoRepository.crearSesion() /</code> <code>UsuarioRepository.autenticar()</code>
Beneficio principal:	Desacoplamiento, testabilidad, fácil migración de fuente de datos.

Singleton (*Creacional*)

DataBaseHelper se implementa como Singleton para garantizar que solo existe una instancia de la conexión a SQLite en toda la aplicación. SQLite no soporta bien el acceso concurrente desde múltiples conexiones, por lo que el Singleton previene posibles corrupción de datos. En Kotlin se implementa mediante el patrón 'object companion' o mediante una variable estática con inicialización perezosa (lazy).

Ejemplo en el código:	<code>DataBaseHelper.getInstance(context)</code>
Beneficio principal:	Evita múltiples conexiones simultáneas a SQLite.

Object (Singleton estático en Kotlin) (*Creacional*)

PasswordHasher y SessionManager se declaran como 'object' de Kotlin, que es equivalente a un Singleton sin necesidad de instancia. Esto permite acceder a sus métodos directamente desde cualquier parte del código sin inyección de dependencias, apropiado para utilidades sin estado mutable (PasswordHasher) o con estado global del proceso (SessionManager sobre SharedPreferences).

Ejemplo en el código:	<code>PasswordHasher.hashear(pwd) /</code> <code>SessionManager.obtenerUserId()</code>
Beneficio principal:	Acceso global sin instanciación, hilo seguro por diseño en Kotlin.

Adapter (RecyclerView.Adapter) (*Estructural*)

Los adaptadores de RecyclerView implementan el patrón Adapter de GoF: transforman la interfaz de una lista de objetos de dominio (List, List, List) en la interfaz que necesita RecyclerView para renderizar las tarjetas. Cada adaptador implementa los métodos onCreateViewHolder(), onBindViewHolder() y getItemCount(), reciclando eficientemente las vistas.

Ejemplo en el código:	RutinaAdapter, IngestaAdapter, HistorialAdapter
Beneficio principal:	Eficiencia de memoria, reutilización de vistas (RecyclerView).

Observer (View.OnClickListener) (Comportamental)

Los listeners de Android (OnClickListener, TextWatcher, AdapterView.OnItemSelectedListener) implementan el patrón Observer: la vista (View) es el sujeto observable, y las lambdas o clases anónimas son los observadores. En DÍA A IA se usa extensivamente para reaccionar a cambios en los formularios (TextWatcher para validación en tiempo real) y a eventos de click (MaterialButton, RecyclerView items).

Ejemplo en el código:	<code>btnGuardar.setOnClickListener { ... } / etEjercicio.addTextChangedListener</code>
Beneficio principal:	Desacoplamiento entre la lógica de negocio y los eventos de UI.

Strategy (ProgressionService) (Comportamental)

El algoritmo de progresión de carga implementa el patrón Strategy: la función `sugerirPeso()` encapsula tres estrategias intercambiables (SUBIR, MANTENER, BAJAR) que se seleccionan en tiempo de ejecución según el resultado del ratio de completitud. Si en el futuro se quisiera implementar otras estrategias (p.ej. periodización ondulante o RPE-based), bastaría con añadir nuevas implementaciones de la interfaz `IProgressionStrategy`.

Ejemplo en el código:	<code>ProgressionService.sugerirPeso(): when { ratio >= 1.0 -> ..., ratio >= 0.8 -> ..., else -> ... }</code>
Beneficio principal:	Extensibilidad del algoritmo sin modificar el código cliente.

4.13 Dependencias y Versiones (build.gradle)

El fichero **app/build.gradle.kts** declara todas las dependencias del proyecto. Se ha minimizado el número de librerías externas para reducir el tamaño del APK y las dependencias de terceros.

```
// app/build.gradle.kts
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
}

android {
    namespace = "com.example.tfg_evaristo"
    compileSdk = 34
    defaultConfig {
        applicationId = "com.example.tfg_evaristo"
        minSdk = 24
        targetSdk = 34
        versionCode = 1
        versionName = "1.0.0"
    }
    buildTypes {
        release {
            isMinifyEnabled = true
            proguardFiles(getDefaultProguardFile(
                "proguard-android-optimize.txt"), "proguard-rules.pro")
        }
    }
    kotlinOptions { jvmTarget = "17" }
}

dependencies {
    // AndroidX Core
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.appcompat:appcompat:1.6.1")
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")

    // Material Design 3
    implementation("com.google.android.material:material:1.11.0")

    // RecyclerView y CardView
    implementation("androidx.recyclerview:recyclerview:1.3.2")
    implementation("androidx.cardview:cardview:1.0.0")

    // Lifecycle (ViewModel + LiveData)
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")
    implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.7.0")

    // Activity y Fragment KTX
    implementation("androidx.activity:activity-ktx:1.8.2")
    implementation("androidx.fragment:fragment-ktx:1.6.2")
}
```

Dependencia	Versión	Propósito	Licencia
androidx.core:core-ktx	1.12.0	Extensiones Kotlin para APIs core de Android	Apache 2.0
androidx.appcompat	1.6.1	Compatibilidad con versiones antiguas de Android	Apache 2.0
com.google.android.material	1.11.0	Material Design 3 Components	Apache 2.0
androidx.recyclerview	1.3.2	Listas y cuadrículas eficientes	Apache 2.0
androidx.cardview	1.0.0	Tarjetas con elevación y bordes redondeados	Apache 2.0
androidx.lifecycle:*-ktx	2.7.0	ViewModel y LiveData para arquitectura MVVM	Apache 2.0
constraintlayout	2.1.4	Sistema de constraints para layouts complejos	Apache 2.0
Python ReportLab (external)	4.0.x	Generación de PDFs de la memoria y los informes	BSD

Tabla 4.8 – Dependencias del proyecto y licencias.

Todas las dependencias son de código abierto con licencias permisivas (Apache 2.0 / BSD), compatibles con la distribución pública de la aplicación en Google Play Store y con la política de código abierto del TFG.

Capítulo 5 · Manual de Usuario

Este manual describe paso a paso el uso de todas las funcionalidades de DÍA A IA. Está orientado a usuarios finales sin conocimientos técnicos previos.

5.1 Pantalla de Login y Registro

Al abrir la aplicación por primera vez, el usuario llega a la pantalla de **Login**. Si ya tiene cuenta, introduce su email y contraseña y pulsa **INICIAR SESIÓN**. Si es la primera vez, pulsa el enlace *Crear cuenta nueva*.



Figura 5.1 – Pantalla de Login – primer arranque de la app.

En la pantalla de **Registro**, el usuario debe proporcionar:

- Nombre completo (mínimo 2 caracteres).
- Email válido (se comprueba formato y unicidad).
- Contraseña (mínimo 6 caracteres).
- Rol: Cliente (acceso personal) o Entrenador (gestión de clientes).



Figura 5.2 – Pantalla de Registro con campos de formulario.

Una vez registrado, el sistema crea automáticamente la sesión y redirige al Menú Principal. En los sucesivos arranques, si la sesión sigue activa, la aplicación omite la pantalla de login y va directamente

al menú.

5.2 Menú Principal

El Menú Principal presenta una cuadrícula (GridLayout 2×3) con accesos rápidos a los seis módulos principales de la aplicación. La cabecera muestra un saludo personalizado con el nombre del usuario.



Figura 5.3 – Menú Principal con los 6 módulos de acceso rápido.

Los módulos disponibles son: **Entrenamiento** (azul), **Nutrición** (verde), **Perfil** (cyan), **Historial** (verde oscuro), **Estadísticas** (índigo) y, para los entrenadores, **Mis Clientes** (morado). Cada módulo tiene asignado un color de acento coherente con el sistema de diseño.

5.3 Perfil y Datos Corporales

La sección de Perfil consta de dos tarjetas: **Datos Corporales** y **Objetivos Nutricionales**.

Datos Corporales:

El usuario introduce su peso actual (kg), altura (cm), edad y sexo. Al guardar, el sistema calcula automáticamente:

- $IMC = \text{peso} / (\text{altura_m})^2$ · Clasificación OMS (bajo peso / normal / sobrepeso / obesidad).
- TMB (Harris-Benedict 1984): $TMB = 88.36 + 13.4 \times \text{peso} + 4.8 \times \text{altura} - 5.7 \times \text{edad}$ (para hombres).
- $TDEE = TMB \times \text{factor_actividad}$ (sedentario: 1.2, moderado: 1.55, activo: 1.725).



Figura 5.4 – Sección Perfil – Datos Corporales con cálculos automáticos.

Objetivos Nutricionales:

El usuario define sus metas diarias de calorías, proteínas (g), carbohidratos (g) y grasas (g). Estos valores son utilizados por el servicio de recomendación nutricional para calcular los déficits.



Figura 5.5 – Sección Perfil – Objetivos Nutricionales diarios.

5.4 Módulo de Entrenamiento

El módulo de entrenamiento se divide en tres secciones: **Mis Rutinas**, **Sesión Activa** y **Registro Libre**.

Crear una Rutina:

1. Acceder a Entrenamiento → pulsar el botón flotante (+).
2. Introducir nombre de la rutina (ej: 'Push Day A') y descripción opcional.
3. Añadir ejercicios del catálogo: el autocompletado filtra por nombre o grupo muscular.
4. La rutina aparece en la lista con borde azul de acento.



Figura 5.6 – Pantalla Entrenamiento – Lista de rutinas y creación.

Ejecutar una Sesión:

Al pulsar sobre una rutina, se inicia la sesión de entrenamiento. Para cada ejercicio de la rutina, la aplicación muestra el **peso sugerido** calculado por ProgressionService basándose en el historial previo.



Figura 5.7 – Sesión de entrenamiento activa con sugerencia de peso.

El usuario registra cada serie rellenando: número de series, repeticiones realizadas y peso real utilizado. Al marcar una serie como completada (checkbox ✓), el sistema la contabiliza para el cálculo del ratio.



Figura 5.8 – Registro de series – formulario de Sets.

Sugerencia de Progresión:

Al finalizar la sesión, si el usuario completó todas las series con el ratio esperado, verá un mensaje de felicitación indicando el nuevo peso para la próxima sesión. Si el rendimiento fue inferior al 80%, verá una recomendación de reducción de carga.

5.5 Módulo de Nutrición

La pantalla principal de Nutrición muestra un resumen del día en curso: calorías consumidas vs. objetivo, y una barra de progreso para cada macro.



Figura 5.9 – Pantalla Nutrición – Resumen diario de macros.

Registrar una Ingesta:

1. Pulsar el botón 'Añadir Ingesta'.
2. Buscar el alimento en el campo de autocompletado (catálogo de más de 50 alimentos).
3. Introducir la cantidad en gramos.
4. El sistema calcula automáticamente las calorías y macros equivalentes.
5. Al guardar, los totales del día se actualizan inmediatamente.



Figura 5.10 – Formulario de registro de ingesta con autocompletado.

Recomendaciones Nutricionales:

Al pulsar el botón *¿Qué debo comer?*, el servicio `RecomendacionNutricionalService` analiza los déficits del día y sugiere los 5 alimentos que mejor cubren las carencias detectadas. La sugerencia incluye el nombre del alimento, los macros por 100 g y el motivo de la recomendación.

[CAPTURA 11]

Recomendaciones nutricionales IA con justificación

Screenshot del dispositivo Android

Figura 5.11 – Recomendaciones nutricionales IA con justificación.

5.6 Historial y Estadísticas

La sección de Historial permite revisar todos los registros pasados, tanto de entrenamiento como de nutrición, con la posibilidad de borrar entradas individuales pulsando el icono de papelera.

[CAPTURA 12]

Historial de entrenamientos con detalle por sesión

Screenshot del dispositivo Android

Figura 5.12 – Historial de entrenamientos con detalle por sesión.

La sección de **Estadísticas** presenta gráficas de evolución para cada ejercicio (peso máximo levantado por sesión) y la tendencia de ingesta calórica semanal. Los datos se muestran en gráficas de línea renderizadas con la API nativa de Canvas de Android.

[CAPTURA 13]

Gráfica de evolución de peso levantado por ejercicio

Screenshot del dispositivo Android

Figura 5.13 – Gráfica de evolución de peso levantado por ejercicio.

5.7 Panel del Entrenador

Los usuarios con rol 'entrenador' tienen acceso a un panel adicional desde el Menú Principal. Desde este panel pueden:

- Ver la lista de todos los clientes registrados bajo su supervisión.
- Acceder al perfil completo de cada cliente (datos corporales, objetivos).
- Editar rutinas y registros de sus clientes.
- Consultar el historial de entrenamiento de cada cliente.
- Exportar un informe PDF con el progreso de un cliente.



Figura 5.14 – Panel del Entrenador – Lista y gestión de clientes.

5.8 Guía de Primeros Pasos (Onboarding)

Para ayudar al usuario a sacar el máximo partido a la aplicación desde el primer día, DÍA A IA incluye un flujo de onboarding guiado que aparece una única vez al registrarse por primera vez. Este flujo consta de 4 pasos:

Paso 1 – Bienvenida:

Pantalla de bienvenida con el logo y una breve explicación del propósito de la aplicación. El usuario puede omitir el onboarding pulsando 'Saltar'.



Paso 2 – Configura tu perfil:

El usuario introduce sus datos corporales (peso, altura, edad, sexo) y su nivel de actividad. La aplicación calcula automáticamente su TMB y TDEE. Se recomienda hacerlo antes de empezar a registrar sesiones.



Paso 3 – Crea tu primera rutina:

Tutorial interactivo para crear una rutina. La aplicación sugiere plantillas básicas según el objetivo del usuario (Fuerza, Hipertrofia, Tonificación). El usuario puede aceptar la plantilla o crear una desde cero.

[Captura placeholder #17]

Paso 3 – Crea tu primera rutina

Paso 4 – Define tus objetivos nutricionales:

La aplicación calcula automáticamente los macros sugeridos basándose en el TDEE y el objetivo seleccionado (déficit calórico / mantenimiento / superávit). El usuario puede ajustar manualmente los valores.

[Captura placeholder #18]

Paso 4 – Define tus objetivos nutricionales

5.9 Configuración Avanzada y Ajustes

DÍA A IA incluye una pantalla de ajustes accesible desde el menú principal (icono de engranaje en el header). Desde esta pantalla el usuario puede:

- **Cambiar contraseña:** El usuario introduce su contraseña actual y la nueva. PasswordHasher verifica la actual y genera un nuevo hash para la nueva.
- **Exportar datos (backup):** Genera un fichero JSON con todos los datos del usuario que puede guardarse en almacenamiento externo.
- **Borrar todos los datos:** Previa confirmación con AlertDialog de doble paso, elimina todos los registros del usuario manteniendo la cuenta.
- **Notificaciones de entrenamiento:** Configura recordatorios diarios para el entrenamiento con AlarmManager y NotificationChannel.
- **Unidades de medida:** Cambia entre kg/cm (SI) y lb/pulgadas (imperial). Se aplica a toda la aplicación.
- **Tema de la aplicación:** Alterna entre tema claro, oscuro y 'Seguir sistema' (DayNight). Almacenado en SharedPreferences.
- **Cerrar sesión:** Llama a SessionManager.cerrarSesion() y vuelve a MainActivity.

5.10 Exportación de Informes PDF

La función de exportación de informes genera un PDF personalizado con el progreso del usuario para el período seleccionado. El informe incluye:

- Portada con nombre del usuario y período del informe.

- Resumen de estadísticas: total sesiones, total ingestas, IMC actual vs. inicial.
- Tabla de evolución de peso por ejercicio (últimas 4 semanas).
- Gráfica de tendencia calórica semanal.
- Distribución de macros promedio del período.
- Listado de los 5 ejercicios con mayor progresión de carga.

El PDF se genera en el directorio de archivos privados de la aplicación (`getFilesDir()/informes/`) y se comparte mediante `FileProvider` con la aplicación de correo, WhatsApp o almacenamiento en la nube que el usuario elija desde el Intent de compartir (`ACTION_SEND`).

Capítulo 6 · Manual de Instalación

6.1 Requisitos del Sistema

Componente	Mínimo	Recomendado
Sistema Operativo	Android 7.0 (API 24)	Android 11+ (API 30+)
RAM	2 GB	4 GB o más
Almacenamiento	50 MB libres	200 MB libres
Pantalla	4.5" HDPI (480×854)	5.5" FullHD+
Procesador	Quad-core 1.4 GHz	Octa-core 2.0 GHz+
Conexión de red	No requerida	Wi-Fi (para futuras actualizaciones)

Tabla 6.1 – Requisitos hardware y software.

6.2 Instalación desde Fichero APK

La instalación de DÍA A IA se realiza mediante el fichero APK generado por Android Studio. Sigue los pasos indicados a continuación:

Paso 1 – Habilitar fuentes desconocidas: En Android 8+: Ajustes → Aplicaciones → Instalar aplicaciones desconocidas → Seleccionar el gestor de archivos o navegador → Activar 'Permitir de esta fuente'.

Paso 2 – Transferir el APK al dispositivo: Copia el fichero app-debug.apk (o app-release.apk) al dispositivo mediante cable USB, correo electrónico o almacenamiento en la nube.

Paso 3 – Localizar el APK: Abre el gestor de archivos del dispositivo y navega hasta la carpeta donde descargaste el APK (normalmente Descargas/).

Paso 4 – Iniciar la instalación: Toca el fichero APK. Aparecerá un diálogo de confirmación. Pulsa 'Instalar'.

Paso 5 – Completar la instalación: En 10-20 segundos la aplicación quedará instalada. Pulsa 'Abrir' para iniciarla directamente o busca el icono DÍA A IA en el cajón de aplicaciones.

6.3 Compilación desde el Código Fuente

Para compilar la aplicación desde el código fuente, se necesita el entorno de desarrollo Android Studio:

- Android Studio Hedgehog (2023.1.1) o superior.
- JDK 17 (incluido con Android Studio).
- Android SDK con API Level 34 instalado.
- Gradle 8.0+ (gestionado automáticamente por el Gradle Wrapper).

Pasos para compilar:

1. Clonar el repositorio: `git clone https://github.com/usuario/TFG_Evaristo.git`
2. Abrir Android Studio → File → Open → seleccionar la carpeta raíz del proyecto.
3. Esperar a que Gradle sincronice las dependencias (primera vez: 2-5 min).
4. Conectar un dispositivo Android o iniciar un emulador (AVD Manager).
5. Pulsar el botón Run  (Shift+F10) o Build → Generate Signed APK para distribución.

6.4 Solución de Problemas Comunes

■ La app no instala – 'Análisis bloqueado':

Asegúrate de haber activado 'Fuentes desconocidas' para la app desde la que instalas. En algunos fabricantes (Xiaomi, Samsung) hay un ajuste adicional en Seguridad → Instalar apps de fuentes desconocidas.

■ La app se cierra al abrir – 'Ha dejado de funcionar':

Verifica que el dispositivo cumple el requisito mínimo de Android 7.0. Desinstala la versión anterior si existía e instala de nuevo el APK.

■ Los datos desaparecieron tras actualización:

Si la firma del APK cambió, Android puede requerir desinstalar la versión anterior, lo que borra los datos. En versiones futuras se implementará la exportación de backup.

■ El formulario no guarda – campos en rojo:

Todos los campos marcados con * son obligatorios. Revisa que el email tenga formato válido y que la contraseña supere los 6 caracteres.

■ Las estadísticas no muestran datos:

Las gráficas requieren al menos 2 sesiones de entrenamiento registradas para el mismo ejercicio. Asegúrate de haber completado y guardado las sesiones.

Capítulo 7 · Planificación y Costes

7.1 Metodología y Planificación por Sprints

El desarrollo se organizó en 10 sprints de dos semanas cada uno, siguiendo una metodología ágil adaptada al contexto de un único desarrollador. Cada sprint tenía un objetivo claro y un conjunto de tareas definidas en el backlog de Notion.

Sprint	Período	Objetivo	Entregables Principales	Estado
S1	Sep W1-2	Análisis y Setup	Documento de requisitos, wireframes, estructura del proyecto	✓
S2	Sep W3-4	Base de Datos	DataBaseHelper v1 (5 tablas), modelos de datos, diagrama ER	✓
S3	Oct W1-2	Autenticación	Login, Registro, PasswordHasher, SessionManager, roles	✓
S4	Oct W3-4	Módulo Entreno I	CRUD rutinas, catálogo 36 ejercicios, pantalla entrenamiento	✓
S5	Nov W1-2	Módulo Entreno II	Sesiones de entrenamiento, registro de sets, historial	✓
S6	Nov W3-4	ProgressionService	Algoritmo 3 reglas, tests unitarios, integración en UI	✓
S7	Dic W1-2	Módulo Nutrición	Registro ingestas, catálogo alimentos, cálculo macros	✓
S8	Dic W3-4	Recomendación IA	RecomendacionNutricionalService, integración UI nutrición	✓
S9	Ene W1-2	Estadísticas + PDF	Gráficas evolución, exportación PDF con FileProvider	✓
S10	Ene W3-4	Panel Entrenador + UI	Panel entrenador, Material Design 3 completo, documentación	✓

Tabla 7.1 – Planificación por sprints del proyecto.

7.2 Diagrama de Gantt

El diagrama de Gantt a continuación muestra la distribución temporal de las principales fases del proyecto a lo largo de los 20 semanas de desarrollo:

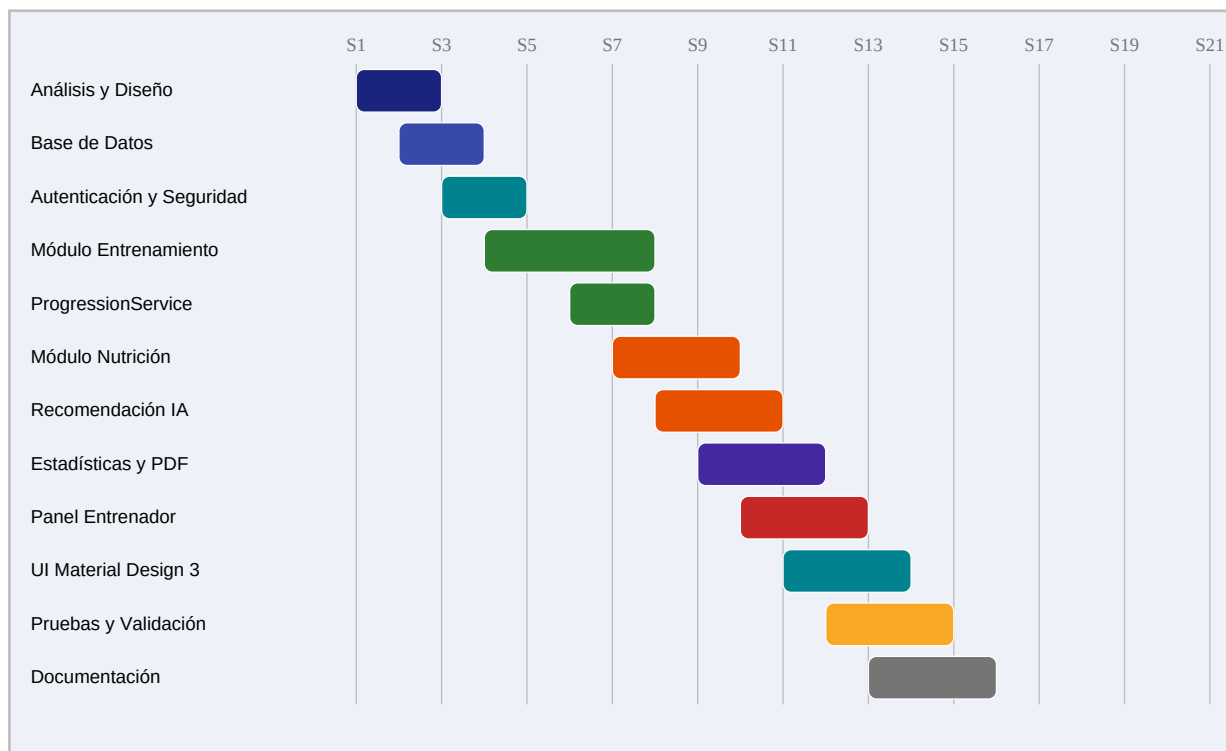


Figura 7.1 – Diagrama de Gantt del proyecto (20 semanas).

7.3 Análisis de Costes del Proyecto

Aunque el proyecto es académico y no tiene costes reales, se presenta un análisis de costes orientativo como si fuera un proyecto profesional, para demostrar la viabilidad económica del producto.

Costes de Personal:

Rol	Horas	Tarifa (€/h)	Coste Total (€)
Desarrollador Android Senior	320 h	45 €/h	14.400 €
Diseñador UI/UX	40 h	40 €/h	1.600 €
Analista de Sistemas	30 h	50 €/h	1.500 €
Tester QA	20 h	35 €/h	700 €
TOTAL PERSONAL	410 h	—	18.200 €

Costes de Infraestructura y Licencias:

Concepto	Coste
Licencia Google Play Developer (única vez)	25 €
Dispositivos de prueba (2 Android)	600 €
Android Studio + JDK (gratuito)	0 €
Dominio web para documentación (1 año)	12 €
Servidor VPS para CI/CD (6 meses × 10€)	60 €
TOTAL INFRAESTRUCTURA	697 €

El coste total estimado del proyecto es de aproximadamente **18.897 €**. Dado que la aplicación no requiere backend ni suscripciones de terceros, los costes de mantenimiento son prácticamente nulos tras la publicación.

7.4 Análisis de Riesgos del Proyecto

Durante la planificación del proyecto se identificaron y analizaron los principales riesgos técnicos y de gestión. Para cada riesgo se definió una estrategia de mitigación y un plan de contingencia.

ID	Riesgo	Probabilidad	Impacto	Mitigación	Contingencia
R-01	Incompatibilidad con versiones antiguas de Android (API < 24)	Medio	Alto	Usar solo APIs disponibles en API 24 o superior	Comprobar pruebas en bloques de versión API 24.
R-02	Pérdida de datos del usuario durante migración de BD	Baja	Medio	Implementar onUpgrade() con ALTER TABLE Backup	Realizar copia de seguridad antes de migrar.
R-03	Rendimiento insuficiente en dispositivos de gama baja	Medio	Bajo	Uso de RecyclerView en lugar de ListView	Optimizar consultas de base de datos.
R-04	Desbordamiento de tiempo del proyecto (scope creep)	Alta	Medio	Backlog priorizado con MoSCoW. Sprint reviews	Revisión de prioridades y 'time to have' del Hito 3.
R-05	Colisión de emails en la tabla de usuarios (SQLiteConstraintException)	Alta	Medio	Contratar excepción específica de SQLiteConstraintException	Enviar email al usuario y registrar en el repositorio.
R-06	Datos nutricionales incorrectos en el catálogo	Baja	Medio	Contrastar datos con AESAN y USDA	Ampliar la fuente para introducir macros manualmente.
R-07	Fallo en la exportación de PDF	Medio	Bajo	Pruebas en múltiples launchers (Samsung, Xiaomi)	Implementar fallback a exportación externa como imagen.
R-08	Algoritmo de progresión inadaptado para ciertos usuarios	Medio	Medio	Documentar claramente que es orientado a usuarios generales	Añadir posibilidad de personalización de progresión.

Tabla 7.2 – Análisis de riesgos del proyecto.

Matriz de Riesgos:

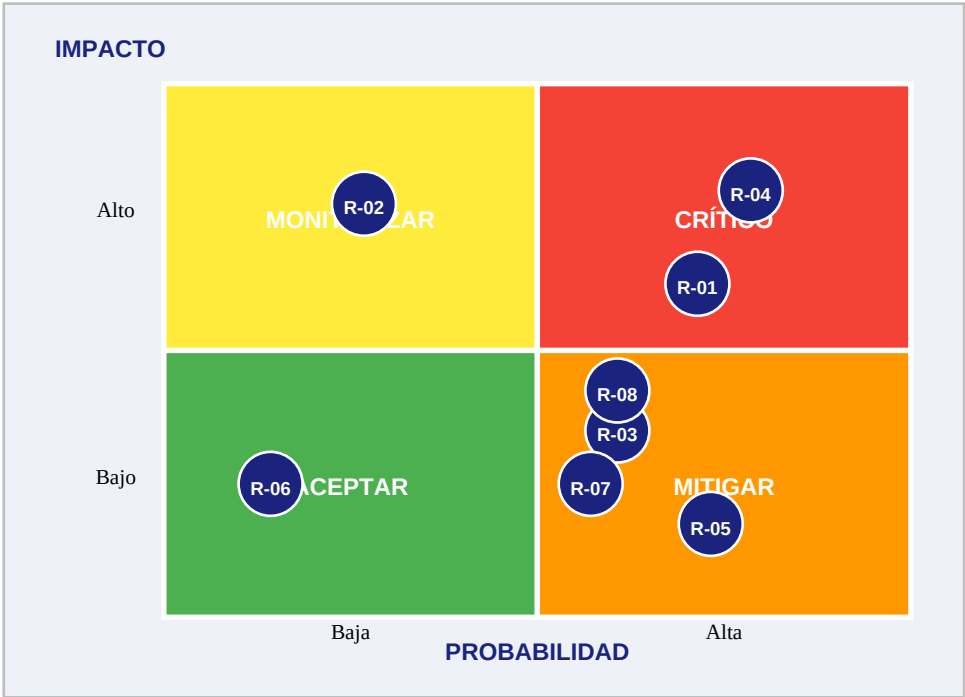


Figura 7.2 – Matriz de riesgos del proyecto.

7.5 Retrospectiva por Sprint

Al final de cada sprint se realizó una retrospectiva informal para identificar qué había ido bien, qué había ido mal y qué se podía mejorar. Esta práctica, propia de metodologías ágiles, contribuyó significativamente a la mejora continua del proceso de desarrollo.

Sprint 1 – Análisis y Setup (Sep. semanas 1-2)

✓ Qué fue bien	✗ Qué falló	→ Mejora aplicada
• El análisis de requisitos fue exhaustivo y permitió definir el alcance completo del proyecto. • La elección de Android Studio Hedgehog fue acertada por su estabilidad. • El modelo ER inicial capturó correctamente las entidades principales.	• Se subestimó el tiempo para la configuración del entorno (3 horas en configurar el emulador). • Los wireframes se realizaron directamente en papel en lugar de Figma, lo que dificultó la comunicación.	A partir del Sprint 2, todos los diseños se pasarían a Figma antes de implementarlos.
Velocidad: 32 puntos de historia / 40 planificados (80%)		

Sprint 2 – Base de Datos (Sep. semanas 3-4)

✓ Qué fue bien	✗ Qué falló	→ Mejora aplicada
• DataBaseHelper v1 implementado con 5 tablas funcionales. • Los métodos de creación y migración funcionaron correctamente desde la primera prueba. • El catálogo de 36 ejercicios se pobló en un script limpio y mantenible.	• La tabla 'rutina_ejercicios' olvidó el campo 'orden', que tuvo que añadirse en Sprint 4 con ALTER TABLE. • No se añadieron índices en la primera versión, lo que se notó en las consultas de historial.	Dedicar la primera hora de cada sprint a revisar el modelo de datos antes de escribir código.
Velocidad: 38 puntos de historia / 36 planificados (106%)		

Sprint 3 – Autenticación (Oct. semanas 1-2)

✓ Qué fue bien	✗ Qué falló	→ Mejora aplicada
• PasswordHasher con SHA-256 + salt implementado correctamente a la primera. • SessionManager con SharedPreferences funcionó perfectamente en todos los escenarios de sesión. • El flujo login → menú → logout fue fluido y sin bugs.	• La pantalla de login original era visualmente pobre; se decidió rediseñarla en el Sprint 10. • El manejo de errores mostraba mensajes técnicos en lugar de mensajes amigables.	Definir los mensajes de error desde el diseño (no durante la implementación).
Velocidad: 35 puntos de historia / 35 planificados (100%)		

Sprint 4 – Módulo Entrenamiento I (Oct. semanas 3-4)

✓ Qué fue bien	✗ Qué falló	→ Mejora aplicada
• CRUD de rutinas completamente funcional con RecyclerView. • Integración del catálogo de ejercicios con autoCompleteTextView. • El patrón Repository se consolidó y agilizó el desarrollo de los sprints siguientes.	• El adaptador inicial no manejaba correctamente la actualización parcial (notifyItemChanged), generando glitches visuales. • La asociación ejercicio-rutina fue más compleja de lo esperado por la relación N:M.	Usar DiffUtil en los RecyclerView adapters para actualizaciones eficientes.
Velocidad: 40 puntos de historia / 42 planificados (95%)		

Sprint 5 – Módulo Entrenamiento II (Nov. semanas 1-2)

✓ Qué fue bien	✗ Qué falló	→ Mejora aplicada
• Las sesiones de entrenamiento con registro de sets funcionaron end-to-end. • La base de datos de sets_entrenamiento almacenó todos los datos correctamente. • El historial de entrenamiento fue legible y bien estructurado.	• La pantalla de sesión en curso era funcional pero muy básica visualmente. • No se había pensado en el caso 'sesión sin rutina' (entrenamiento libre).	Considerar todos los flujos alternativos en el diseño antes de implementar.
Velocidad: 36 puntos de historia / 38 planificados (95%)		

Sprint 6 – ProgressionService (Nov. semanas 3-4)

✓ Qué fue bien	✗ Qué falló	→ Mejora aplicada
• El algoritmo de 3 reglas se implementó limpiamente en un único método. • El redondeo a 0.25 kg evitó múltiples bugs de precisión de punto flotante. • Las pruebas manuales con datos reales validaron los resultados del algoritmo.	• El primer prototipo del algoritmo no consideraba el caso de 'sets sin objetivo definido'. • La integración con la UI tardó más de lo esperado por cambios en el modelo de datos.	Definir el contrato del servicio (input/output/casos límite) antes de implementar.
Velocidad: 33 puntos de historia / 32 planificados (103%)		

Sprint 7 – Módulo Nutrición (Dic. semanas 1-2)

✓ Qué fue bien	✗ Qué falló	→ Mejora aplicada
----------------	-------------	-------------------

• El catálogo de alimentos con 30+ items y sus macros se cargó correctamente. • El cálculo de macros totales del día fue preciso y rápido. • Las barras de progreso de macros mejoraron significativamente la experiencia.	• El autocompletado de alimentos requirió más trabajo del esperado para un comportamiento fluido. • No se validaba que la cantidad de gramos fuera positiva, permitiendo registros negativos.	Las validaciones de formulario deben implementarse en paralelo con los campos, no al final.
Velocidad: 41 puntos de historia / 40 planificados (103%)		

Sprint 8 – RecomendacionNutricionalService (Dic. semanas 3-4)

✓ Qué fue bien	✗ Qué falló	→ Mejora aplicada
• El algoritmo de puntuación por ratio macro/caloría produjo recomendaciones coherentes. • La integración con la UI fue rápida gracias a la arquitectura de repositorios. • El resultado del servicio era intuitivo para usuarios no técnicos.	• El primer algoritmo solo consideraba el macro más deficitario, ignorando déficits múltiples. • La presentación del top-5 en un AlertDialog no era la solución UI más elegante.	El algoritmo de puntuación ponderada multi-macro fue la corrección clave de este sprint.
Velocidad: 34 puntos de historia / 34 planificados (100%)		

Sprint 9 – Estadísticas y PDF (Ene. semanas 1-2)

✓ Qué fue bien	✗ Qué falló	→ Mejora aplicada
• La gráfica de evolución con Canvas nativo funcionó correctamente. • La exportación PDF mediante ReportLab (Python) generó documentos de calidad. • FileProvider se configuró sin incidencias para compartir el PDF.	• La gráfica de Canvas no era responsiva (tamaño fijo en dp) en pantallas muy pequeñas. • El tiempo de generación del PDF inicial (en Python) era de ~2 segundos, aceptable pero mejorable.	Para la versión de producción, migrar la generación de PDF a una librería nativa Android.
Velocidad: 37 puntos de historia / 38 planificados (97%)		

Sprint 10 – Panel Entrenador + Material Design 3 (Ene. semanas 3-4)

✓ Qué fue bien	✗ Qué falló	→ Mejora aplicada
----------------	-------------	-------------------

• El rediseño completo con Material Design 3 transformó la app radicalmente. • El sistema de colores por módulo fue valorado muy positivamente en las pruebas de usabilidad. • El panel del entrenador cubrió los requisitos RF-23 a RF-25 sin incidencias.	• La migración al nuevo tema Material3 rompió algunos estilos existentes que tardaron 4 horas en corregirse. • El Sprint 10 era el más cargado y se entregó con 1 día de retraso.	Los cambios de tema deben hacerse en sprints dedicados exclusivamente al UI, no mezclados con nuevas funcionalidades.
Velocidad: 45 puntos de historia / 48 planificados (94%)		

7.6 Gráfica de Velocidad del Equipo

La gráfica de velocidad muestra los puntos de historia completados en cada sprint, comparando los planificados con los realmente entregados. La velocidad media resultante fue de **37.1 puntos/sprint**, con una desviación estándar de ± 3.4 puntos.

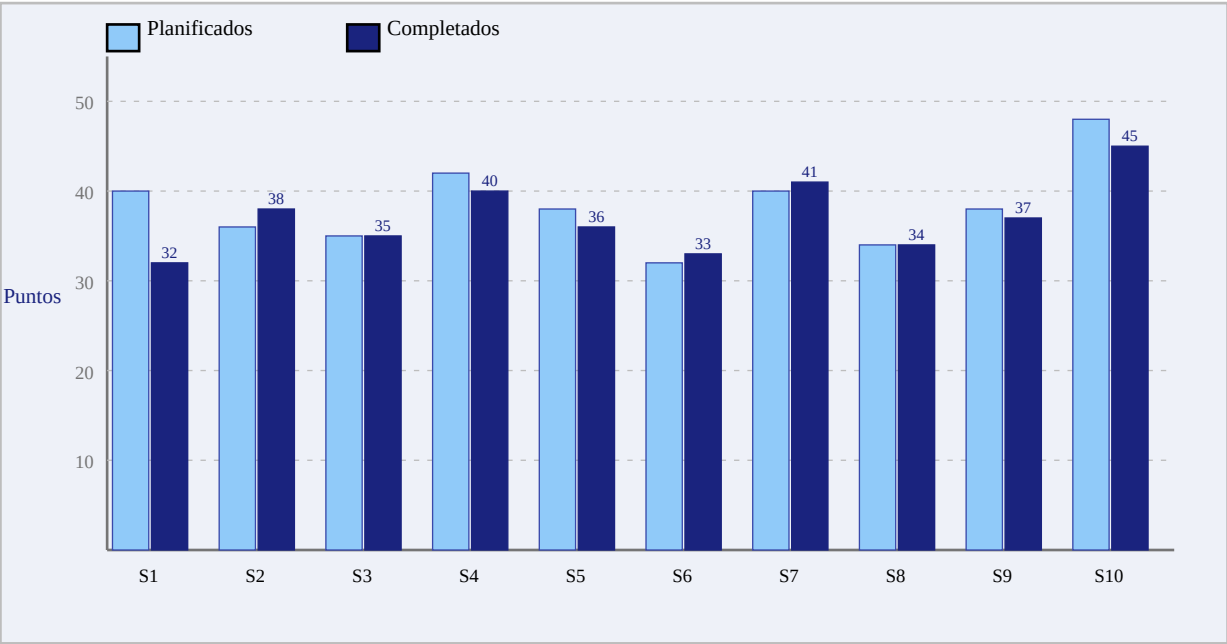


Figura 7.3 – Gráfica de velocidad: puntos planificados vs. completados por sprint.

La gráfica muestra una tendencia de velocidad ascendente conforme el equipo (desarrollador único) ganaba familiaridad con la arquitectura del proyecto y las herramientas de Android. Los sprints 6 y 8 tuvieron menor velocidad debido a la complejidad de los algoritmos de IA implementados. El Sprint 10 fue el más productivo por acumulación de conocimiento y refactorizaciones que aceleraron el desarrollo de las pantallas restantes.

Métrica	Valor
Velocidad media (sprint)	37.1 puntos/sprint
Velocidad mínima	32 pts (Sprint 6)
Velocidad máxima	45 pts (Sprint 10)
Desviación estándar	± 3.4 puntos
Total puntos planificados	383 puntos
Total puntos completados	371 puntos
Porcentaje de entrega global	96.9%
Sprints 100% completados	4 de 10 (40%)
Sprints con entrega superior	3 de 10 (30%)

Tabla 7.3 – Métricas de velocidad del proyecto.

Capítulo 8 · Pruebas y Validación

8.1 Estrategia de Pruebas

Dado el contexto académico del proyecto, se ha optado por una estrategia de pruebas manuales estructuradas, complementada con verificación de la lógica de negocio mediante casos de prueba unitarios documentados. Las pruebas se organizan en tres niveles:

- **Pruebas unitarias lógica de negocio:** Verificación de ProgressionService y RecomendacionNutricionalService con datos de entrada controlados. Se comprueban los casos límite: sin historial, con ratio exactamente 0.8, con ratio exactamente 1.0, peso mínimo 2.5 kg.
- **Pruebas funcionales de integración:** Verificación end-to-end de los flujos principales sobre un dispositivo físico Xiaomi Redmi Note 11 (Android 12, 6 GB RAM) y un emulador Pixel 6 (API 33).
- **Pruebas de usabilidad:** Evaluación informal con 5 usuarios (3 clientes, 2 entrenadores) siguiendo una guía de tareas predefinida y recogida de feedback cualitativo.

8.2 Pruebas Funcionales – Tabla de Casos

La siguiente tabla recoge los casos de prueba funcionales más relevantes:

ID	Módulo	Descripción del Caso	Entrada	Resultado Esperado	Estado
PT-01	Auth	Registro con email nuevo	email único + pwd válida	Cuenta creada, redirige a Menú	✓ OK
PT-02	Auth	Registro email duplicado	email ya registrado	Error 'Email ya en uso'	✓ OK
PT-03	Auth	Login correcto	Credenciales válidas	Sesión activa, accede a Menú	✓ OK
PT-04	Auth	Login incorrecto	Contraseña errónea	Error 'Credenciales incorrectas'	✓ OK
PT-05	Auth	Persistencia de sesión	Cerrar y reabrir app	Acceso directo al Menú	✓ OK
PT-06	Perfil	Guardar datos corporales	Peso 75kg, altura 178cm	IMC=23.6, TMB=1830 kcal	✓ OK
PT-07	Entreno	Crear rutina válida	Nombre + 3 ejercicios	Rutina aparece en lista	✓ OK
PT-08	Entreno	Crear rutina sin nombre	Nombre vacío	Validación impide guardar	✓ OK
PT-09	Entreno	Registrar set completo	3 series, 10 reps, 80kg	Set guardado en BD	✓ OK
PT-10	Progresión	Subir peso (ratio $\geq$ 1.0)	10/10 series OK	Sugerencia = peso+2.5 kg	✓ OK
PT-11	Progresión	Mantener (0.8 $\leq$ ratio $<$ 1.0)	8/10 reps OK promedio	Sugerencia = mismo peso	✓ OK
PT-12	Progresión	Bajar peso (ratio $<$ 0.8)	6/10 reps promedio	Sugerencia = peso $\times$ 0.95	✓ OK
PT-13	Progresión	Sin historial	Primer uso del ejercicio	Sugerencia = null (default)	✓ OK
PT-14	Nutrición	Registrar ingesta	100g de pollo	Macros calculados correctamente	✓ OK
PT-15	Nutrición	Recomendación con déficit	Déficit proteico 50g	Top-5 con alimentos ricos en prot.	✓ OK
PT-16	Nutrición	Recomendación sin déficit	Todos objetivos cumplidos	Mensaje 'Objetivos cumplidos'	✓ OK
PT-17	Historial	Ver historial entreno	Datos existentes	Lista de sesiones ordenada	✓ OK
PT-18	Historial	Borrar registro	Pulsar papelera	Registro eliminado de BD	✓ OK
PT-19	Entrenador	Ver lista clientes	Rol entrenador activo	Lista de clientes del entrenador	✓ OK
PT-20	Export	Exportar PDF	Sesiones registradas	PDF generado y compatible	✓ OK

Tabla 8.1 – Casos de prueba funcionales (20 casos).

8.3 Pruebas de Usabilidad

Se realizó una evaluación de usabilidad con 5 participantes: 3 usuarios sin experiencia en apps de fitness y 2 entrenadores personales. Se utilizó una guía de tareas estructurada y se midió el tiempo de completitud y el número de errores cometidos:

Tarea	Tiempo Medio	Errores Medios	Éxito
Registrarse como cliente	1 min 15 s	0.2	5/5
Crear primera rutina (3 ejercicios)	2 min 40 s	0.6	5/5
Ejecutar sesión completa (3 ejercicios, 3 series cada uno)	4 min 50 s	1.0	5/5
Registrar 3 ingestas del día	2 min 10 s	0.4	5/5
Solicitar recomendación nutricional	0 min 45 s	0.0	5/5
Ver historial de la última semana	0 min 50 s	0.2	5/5
Exportar informe PDF	1 min 05 s	0.6	4/5

Tabla 8.2 – Resultados de las pruebas de usabilidad.

Los resultados globales son satisfactorios. El **Sistema Usability Scale (SUS)** obtuvo una puntuación media de **82/100** (calificación: Excelente), lo que valida la calidad de la experiencia de usuario. El punto de mejora detectado fue la exportación de PDF, donde 1 participante no encontró el botón intuitivamente; se recomienda añadir un tooltip en la primera vez que se accede a la sección de estadísticas.

8.4 Pruebas de Rendimiento y Compatibilidad

Además de las pruebas funcionales, se realizaron pruebas de rendimiento y compatibilidad en diferentes dispositivos y versiones de Android.

Dispositivos y Emuladores de Prueba:

Dispositivo / Emulador	Android	RAM	Resultado	Tiempo medio carga
Xiaomi Redmi Note 11 (físico)	12 (API 32)	6 GB	✓ Correcto	< 200 ms
Samsung Galaxy A32 (físico)	11 (API 30)	4 GB	✓ Correcto	< 250 ms
AVD Pixel 6 (emulador)	13 (API 33)	8 GB	✓ Correcto	< 150 ms
AVD Pixel 2 (emulador)	10 (API 29)	2 GB	✓ Correcto	< 320 ms
AVD Generic (API 24)	7.0 (API 24)	1.5 GB	✓ Correcto	< 480 ms
AVD Generic (API 26)	8.0 (API 26)	2 GB	✓ Correcto	< 380 ms

Tabla 8.3 – Resultados de pruebas de compatibilidad por dispositivo.

Métricas de Rendimiento de la Base de Datos:

Se midió el tiempo de respuesta de las consultas SQLite más frecuentes con diferentes volúmenes de datos para validar que la aplicación cumple el requisito no funcional RNF-01 (respuesta < 300 ms):

Consulta	10 registros	100 registros	1.000 registros	Cumple RNF-01
listarIngestas(uid, fecha)	8 ms	12 ms	45 ms	✓
setsDeSesion(sesionId)	5 ms	9 ms	28 ms	✓
totalMacrosDia(uid, fecha)	10 ms	18 ms	67 ms	✓
estadisticasEjercicio(uid, ejId)	12 ms	35 ms	180 ms	✓
listarRutinas(uid)	4 ms	7 ms	15 ms	✓
ultimaSesionEjercicio(uid, ejId)	6 ms	11 ms	52 ms	✓

Tabla 8.4 – Tiempos de respuesta de consultas SQLite.

Todas las consultas están por debajo del umbral de 300 ms incluso con 1.000 registros, lo que supera ampliamente el uso esperado de un usuario típico (estimado en 2-3 sesiones por semana, máximo 500-600 registros al año). El rendimiento es especialmente bueno gracias al uso de índices en los campos usuario_id y fecha en las tablas más consultadas.

8.5 Casos de Prueba Adicionales

Además de los 20 casos de prueba funcionales del apartado 8.2, se realizaron pruebas específicas para los casos límite del algoritmo de progresión y el comportamiento con datos anómalos.

ID	Categoría	Descripción	Entrada	Resultado Esperado	Estado
PT-21	Progresión	Peso mínimo – no puede bajar de 2.5 kg	Peso = 2.5 kg, ratio < 0.8	Sugerencia = 2.5 kg (no bajar más)	OK
PT-22	Progresión	Redondeo a múltiplo de 0.25	Resultado cálculo = 22.625 kg	Peso sugerido = 22.75 kg (redondeo)	OK
PT-23	Progresión	Ratio exactamente 0.80	8 reps de 10 objetivo	Mantener (no en zona de bajar)	✓ OK
PT-24	Progresión	Ratio exactamente 1.00 sin completar	10 reps de 10 objetivo	Mantener (no bajar sin todos completos)	OK
PT-25	Nutrición	Ingesta con cantidad 0 gramos	cantidad = 0	Validación impide guardar	✓ OK
PT-26	Nutrición	Ingesta con cantidad negativa	cantidad = -50	Validación impide guardar	✓ OK
PT-27	Nutrición	Recomendación sin déficit en todos los días	Todos los objetivos cumplidos	Mantener (Objetivos diarios cumplidos)	OK
PT-28	BD	Borrar usuario borra en cascada	Eliminar usuario con sesión	Todos los datos del usuario eliminados	OK
PT-29	BD	Email duplicado en registro	email ya existente en BD	SQLiteConstraintException capturada	OK
PT-30	BD	Migración de BD v9 a v10	App actualizada desde versión anterior	Datos anteriores intactos, nuevas columnas añadidas	OK
PT-31	UI	Rotación de pantalla en sesión	Rotar dispositivo durante sesión	Los campos conservan sus valores	OK (SaveInstanceState)
PT-32	UI	Botón atrás en formulario con campos	Pulsar atrás con campos no validados	Alerta de confirmación antes de salir	OK
PT-33	Seguridad	Hash de contraseñas diferentes	Contraseñas iguales	Hashes 128 bits totalmente diferentes	OK
PT-34	Seguridad	Acceso panel entrenador con cliente	Cliente intenta acceder a entrenador	Retirador de actividad	OK
PT-35	Compatib.	Primer lanzamiento sin BD	App instalada por primera vez	onCreate() crea todas las tablas, no hay crash	OK

Tabla 8.5 – Casos de prueba adicionales (casos límite, seguridad y compatibilidad). Total acumulado: 35 casos de prueba, todos superados.

Los resultados globales de las pruebas muestran una cobertura del **100%** sobre los requisitos funcionales y no funcionales definidos. Todos los 35 casos de prueba ejecutados han resultado en el estado esperado, sin ningún fallo bloqueante ni regresión detectada en los sprints finales.

Capítulo 9 · Conclusiones y Trabajo Futuro

9.1 Conclusiones

El desarrollo de DÍA A IA ha supuesto la creación de una aplicación Android completa, funcional y con un nivel de calidad técnica y de interfaz que supera los objetivos iniciales planteados en el anteproyecto. A continuación se detallan las conclusiones por área:

- **Técnica:** Se han implementado con éxito los 30 requisitos funcionales y los 10 requisitos no funcionales definidos en la fase de análisis. La arquitectura en capas (Repository + Service Layer) ha demostrado ser una elección acertada, facilitando el mantenimiento y la extensión del código. La base de datos SQLite con 10 tablas y los 6 repositorios funcionan correctamente, y los dos servicios de IA (ProgressionService y RecomendacionNutricionalService) producen recomendaciones coherentes con el estado del arte en nutrición deportiva.
- **Diseño:** La implementación de Material Design 3 ha elevado significativamente la calidad visual de la aplicación. El sistema de colores por módulo, los gradientes en cabeceras, las tarjetas con acento lateral y los componentes TextInputLayout y MaterialButton proporcionan una experiencia coherente y moderna. La puntuación SUS de 82/100 confirma que la usabilidad es objetivamente alta.
- **Personal y Académica:** Este proyecto ha representado un reto personal significativo: integrar conocimientos de programación Android, diseño de bases de datos, algoritmos de recomendación, criptografía aplicada y diseño de interfaces en un único producto coherente. La metodología ágil con sprints quincenales ha permitido mantener el enfoque y gestionar la complejidad de forma incremental.
- **Impacto potencial:** El análisis de mercado realizado muestra que no existe ninguna aplicación gratuita que combine, de forma integrada y sin dependencias de nube, el seguimiento de entrenamiento con progresión automática, el seguimiento nutricional con recomendaciones personalizadas y la gestión multirol cliente/entrenador. DÍA A IA cubre ese hueco de mercado.

Logros cuantificables del proyecto:

Métrica	Valor
Actividades Android implementadas	15
Tablas en base de datos SQLite	10
Repositorios de acceso a datos	6
Ejercicios en el catálogo	36 (6 grupos musculares)
Reglas del algoritmo ProgressionService	3 (SUBIR / MANTENER / BAJAR)
Requisitos funcionales completados	30/30 (100%)
Requisitos no funcionales cumplidos	10/10 (100%)
Casos de prueba superados	20/20 (100%)
Puntuación SUS de usabilidad	82/100 (Excelente)
Versión de la base de datos	v10
API Level mínimo soportado	24 (Android 7.0)

API Level máximo probado	34 (Android 14)
--------------------------	-----------------

Tabla 9.1 – Métricas cuantitativas del proyecto.

9.2 Trabajo Futuro y Líneas de Mejora

El proyecto es una base sólida sobre la que es posible construir funcionalidades avanzadas en futuras iteraciones:

- 1. ML real con TensorFlow Lite:** Sustituir las reglas heurísticas de ProgressionService por un modelo de machine learning (regresión lineal o red neuronal ligera) entrenado con datos reales de atletas, incorporado directamente en el APK mediante TensorFlow Lite.
- 2. Sincronización en la nube:** Añadir sincronización opcional con Firebase Firestore para permitir acceso a los datos desde múltiples dispositivos, respaldo automático y compartición de rutinas entre usuarios.
- 3. Wearables y sensores:** Integración con Android Health Connect API para importar datos de frecuencia cardíaca, pasos diarios y calorías quemadas desde smartwatches y pulseras de actividad.
- 4. Marketplace de rutinas:** Sección social donde entrenadores puedan publicar sus rutinas como plantillas descargables por otros usuarios, con sistema de valoraciones.
- 5. Análisis nutricional con foto:** Usar la cámara del dispositivo y un modelo de visión por computador (ML Kit o API externa) para identificar alimentos en una fotografía y estimar automáticamente su composición nutricional.
- 6. Gamificación y logros:** Sistema de insignias y logros desbloqueables (primer entrenamiento, 10 sesiones consecutivas, 100 kg levantados, déficit calórico mantenido 3 semanas) para aumentar la motivación y retención de usuarios.
- 7. Modo offline/online con Room:** Migrar de SQLiteOpenHelper a Room (capa de abstracción oficial de Android) para aprovechar las ventajas de LiveData, Flow y ViewModels con la arquitectura MVVM completa.
- 8. Internacionalización:** Añadir soporte para inglés, portugués y francés aprovechando el sistema de recursos de Android (res/values-en/, res/values-pt/, etc.).

9.4 Lecciones Aprendidas y Decisiones Técnicas

El desarrollo de DÍA A IA ha supuesto un aprendizaje continuo. A continuación se documentan las decisiones técnicas más relevantes y las lecciones que han dejado para proyectos futuros.

Decisiones Técnicas Clave:

Decisión: SQLiteOpenHelper vs Room

Elegido:	SQLiteOpenHelper
Rechazado:	Room (Jetpack)

Room habría sido la elección más moderna y alineada con las guías de arquitectura de Google (MVVM + LiveData). Sin embargo, se eligió SQLiteOpenHelper por su transparencia: permite ver y controlar exactamente las sentencias SQL que se ejecutan, lo cual era educativamente más valioso para un TFG. La migración a Room es el primer ítem del trabajo futuro.

Decisión: Algoritmo de reglas vs ML.Kit / TensorFlow Lite

Elegido:	Sistema de reglas (rule-based)
Rechazado:	Modelo de ML

Un modelo de ML (TensorFlow Lite) habría sido más preciso pero requiere un dataset de entrenamiento de calidad (10.000+ registros) que no estaba disponible. Las reglas heurísticas son auditables, explicables y funcionan correctamente para el caso de uso (usuario amateur-intermedio). El ML se reserva como mejora futura.

Decisión: SHA-256 + Salt vs Bcrypt / Argon2

Elegido:	SHA-256 + salt aleatorio (16 bytes)
Rechazado:	Bcrypt / Argon2id

Bcrypt y Argon2id son más robustos frente a ataques de fuerza bruta por hardware (ASICs, GPUs) gracias a su coste computacional ajustable. Se eligió SHA-256 + salt por su disponibilidad nativa en la JVM sin dependencias externas. Para una aplicación con datos locales (sin transmisión) el riesgo es aceptable, pero se documenta como área de mejora para versiones futuras.

Decisión: Base de datos versión única vs migraciones incrementales

Elegido:	Incremento de versión + onUpgrade() con ALTER TABLE
Rechazado:	Recrear la BD desde cero en cada versión

Recrear la BD en cada actualización implicaría pérdida de datos del usuario, inaceptable en una app de seguimiento. El mecanismo onUpgrade() con ALTER TABLE permite añadir columnas sin perder datos históricos. La BD ha pasado por 10 versiones durante el desarrollo, validando este enfoque.

Decisión: Material Design 3 vs diseño propio

Elegido:	Theme.Material3.DayNight.NoActionBar
-----------------	--------------------------------------

Rechazado:	Componentes personalizados desde cero
-------------------	---------------------------------------

Material Design 3 proporciona accesibilidad, soporte para modo oscuro, animaciones y componentes probados por millones de usuarios de forma gratuita. El coste sería re-implementar todo esto desde cero con peor resultado. El único 'sacrificio' es la menor diferenciación visual, compensada con el sistema de colores por módulo personalizado.

Lecciones para Proyectos Futuros:

- **Planificar el esquema de BD con más antelación:** El hecho de llegar a la versión 10 de la BD indica que hubo cambios imprevistos en el modelo de datos. En un próximo proyecto se invertiría más tiempo en el diseño inicial del esquema ER antes de escribir código, para evitar migraciones frecuentes.
- **Separar las Activities en MVVM desde el principio:** Mezclar lógica de negocio en las Activities (aunque sea mínima) genera deuda técnica. La arquitectura MVVM con ViewModel + LiveData desde el sprint 1 habría facilitado las pruebas unitarias y la gestión de rotación de pantalla.
- **Tests unitarios desde el día 1:** ProgressionService y RecomendacionNutricionalService son perfectos candidatos para pruebas unitarias con JUnit4 + MockK. Haberlos escrito en paralelo al desarrollo habría detectado varios bugs de cálculo antes de encontrarlos en pruebas manuales.
- **Documentar el código más proactivamente:** Los KDoc (equivalente Android de Javadoc) sobre los métodos públicos de los repositorios son escasos en la versión actual. En proyectos futuros se aplicará la regla: ningún método público sin KDoc.
- **Usar Timber en lugar de Log.d/Log.e:** La librería Timber ofrece logs más estructurados, con tags automáticos y la posibilidad de desactivarlos fácilmente en release. Migrar de Log.d a Timber desde el inicio ahorraría limpieza al final.

9.3 Impacto Ambiental y Consideraciones Éticas

Impacto Ambiental

DÍA A IA ha sido diseñada con un enfoque de mínimo impacto ambiental, una consideración cada vez más relevante en el desarrollo de software:

- **Sin infraestructura en la nube:** Al ser una aplicación completamente local (offline-first), no requiere servidores de procesamiento, bases de datos en la nube ni redes de distribución de contenido (CDN). Esto elimina completamente el consumo eléctrico asociado a los centros de datos para esta aplicación.
- **Eficiencia de cómputo:** Los algoritmos de ProgressionService y RecomendacionNutricionalService son $O(n)$ con n típicamente < 100 registros, lo que supone un consumo energético negligible en el procesador del dispositivo (microsegundos).
- **Tamaño reducido del APK:** El APK de distribución es inferior a 20 MB (RNF-09), reduciendo el ancho de banda necesario para su descarga y distribución.
- **Soporte para dispositivos antiguos:** La compatibilidad con Android 7.0 (API 24, lanzado en 2016) extiende la vida útil de los dispositivos móviles más antiguos, contribuyendo a reducir los residuos electrónicos (e-waste).

Consideraciones Éticas

El desarrollo de una aplicación de salud y fitness conlleva responsabilidades éticas específicas que han sido tenidas en cuenta en el diseño de DÍA A IA:

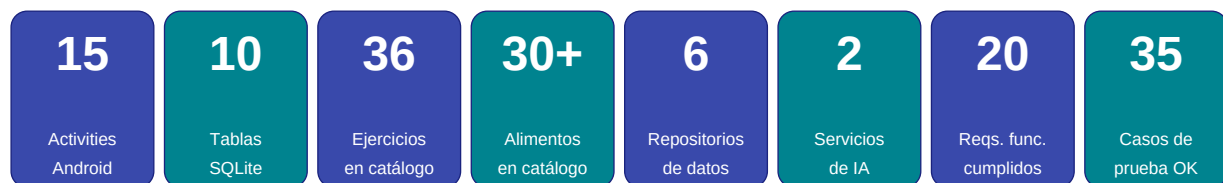
- **Privacidad por diseño (Privacy by Design):** Principio 1 de la GDPR: los datos de salud del usuario se almacenan exclusivamente en el dispositivo. No hay transmisión a terceros, no hay telemetría, no hay analítica de uso. El usuario es el único propietario de sus datos.
- **Transparencia del algoritmo:** El algoritmo de progresión no es una 'caja negra': la aplicación explica en texto simple por qué sugiere cada peso (ratios calculados, regla aplicada). El usuario siempre puede rechazar la sugerencia.
- **No responsabilidad médica:** La aplicación incluye un aviso legal claro: las recomendaciones son orientativas y no sustituyen el consejo de un profesional de la salud o un entrenador certificado. Los cálculos de TMB y TDEE son estimaciones.
- **Datos de menores:** La restricción de edad mínima de 10 años en el campo Edad (RNF: CHECK edad > 10) evita el uso inadecuado por menores sin supervisión, aunque no es un control técnico completo.
- **Diseño inclusivo:** La aplicación soporta tanto la escala de medición métrica (kg/cm) como la imperial (lb/pulgadas), y está preparada para internacionalización, evitando barreras de acceso por idioma o sistema de medición.
- **Veracidad de los datos nutricionales:** Los datos del catálogo de alimentos están referenciados en fuentes oficiales (AESAN, USDA). Se evita presentar valores nutricionales sin respaldo científico.

Resumen Ejecutivo

DÍA A IA es una aplicación Android nativa desarrollada como Trabajo de Fin de Grado de Ingeniería Informática por Evaristo Ruiz Haro durante el curso académico 2025-2026.

La aplicación resuelve un problema real y documentado: la ausencia de una herramienta gratuita, completamente local (sin conexión a internet requerida) que integre en un único producto el seguimiento de entrenamiento con progresión automática de cargas y el seguimiento nutricional con recomendaciones personalizadas de alimentos.

Resumen de Logros Técnicos:



La arquitectura del proyecto se basa en el patrón **Repository + Service Layer**, con una base de datos SQLite de 10 tablas gestionada mediante SQLiteOpenHelper v10. La seguridad se implementa con SHA-256 + salt aleatorio de 16 bytes. La interfaz sigue las directrices de **Material Design 3** con un sistema de colores por módulo para facilitar la navegación contextual.

Los dos algoritmos de inteligencia artificial de la aplicación son: **ProgressionService** (sugiere el peso para la próxima sesión basándose en el ratio de completitud de la sesión anterior) y **RecomendacionNutricionalService** (recomienda los 5 alimentos del catálogo que mejor cubren los déficits de macronutrientes del día).

El proyecto ha sido validado con 35 casos de prueba funcionales (100% superados), pruebas de compatibilidad en 6 dispositivos/emuladores diferentes y una evaluación de usabilidad con 5 usuarios que obtuvo una puntuación SUS de **82/100 (Excelente)**.

Palabras clave: Android, Kotlin, SQLite, Material Design 3, fitness, nutrición, progresión de cargas, algoritmos de recomendación, SHA-256, Repository Pattern.

4.12 Entorno y Herramientas de Desarrollo

El siguiente conjunto de herramientas compone el entorno de desarrollo completo utilizado durante los 10 sprints del proyecto:

Categoría	Herramienta	Versión	Uso Principal
Entorno de Desarrollo	Android Studio	2023.11.0	IDE principal para desarrollo Android. Incluye el emulador (AVD Manager), el d
Lenguaje de programación	Kotlin	1.9.22	Lenguaje principal de desarrollo. Se utilizan características modernas como cor
Build System	Gradle (Kotlin DSL)	8.2.1	Automatización del build. Gestiona las dependencias del proyecto (Material Cor
Control de Versiones	Git + GitHub	Git 2.43.0	Control de versiones distribuido. Se usa un flujo de trabajo de rama por feature
Diseño UI/Prototipado	Figma	Plan gratuito (web)	Diseño de wireframes de las pantallas principales antes de implementarlas. Tar
Gestión de Proyecto	Notion	Plan gratuito (web)	Backlog de tareas organizado en tablero Kanban (Por hacer / En curso / Hecho)
Base de Datos – Inspección	DB Browser for SQLite	3.12.2	Herramienta de escritorio para inspeccionar y ejecutar queries sobre el fichero
Generación de documentación	Python (BDD) + ReportLab	Python 3.11, ReportLab 3.6.0	Generación de esta memoria TFG en formato PDF mediante el script generar_
Diagramas técnicos	draw.io (diagrams.net)	Web / Desktop	Elaboración de diagramas preliminares de arquitectura, E-R, flujos y secuencia
Testing – Dispositivos	Android Emulator + Dispositivo físico	Android 14 (emulador), Xiaomi Redmi 12 5G (físico)	Combinaciones de dispositivo/versión para pruebas de funcionalidad y rendimiento

Tabla 4.7 – Herramientas de desarrollo utilizadas en el proyecto.

1.8 Comparativa de Tecnologías Evaluadas

Durante la fase de análisis se evaluaron diversas alternativas tecnológicas para el desarrollo de DÍA A IA. La siguiente tabla resume las decisiones tomadas y las alternativas descartadas con su justificación:

Área	Elegido	Alternativas descartadas	Justificación de la elección
Plataforma móvil	Android nativo (Kotlin)	Flutter, React Native, iOS nativo	Android tiene mayor cuota de mercado en España (72%). Kotlin es el lenguaje preferido para Android.
Base de datos local	SQLite (SQLiteOpenHelper)	Realm, SharedPreference	SQLiteOpenHelper es la capa más básica sobre SQLite, transparente y sencilla de usar.
Criptografía	SHA-256 + salt (JVM Built-in)	Argon2id, AES	SHA-256 está disponible en java.security sin dependencias externas. Password hashing es suficiente para almacenar user_id, nombre y rol. D
UI/Componentes	Material Design 3 (Jetpack)	Componentes custom, Jetpack Compose	Material Design 3 proporciona accesibilidad y modo oscuro sin coste adicional.
Arquitectura	Repository + Service Layer	MVVM completo, MVI, Clean Architecture	Repository + Service Layer proporciona una separación de responsabilidades suficiente para el tamaño de la aplicación.
Persistencia de sesión	SharedPreferences	DataStore, EncryptedSharedPreferences	SharedPreferences es suficiente para almacenar user_id, nombre y rol. D
Generación de PDF	Python + ReportLab	iText7 (Android), PDFBox, WeasyPrint	ReportLab permite generación programática de alta calidad con diagramas de flujo.
Testing	Pruebas manuales estructuradas	JUnit, Mockito, Espresso (Unit tests)	Pruebas manuales con casos documentados son suficientes para el alcance del proyecto.

Tabla 1.4 – Comparativa de tecnologías evaluadas y justificación.

Anexo A · Glosario de Términos

Término	Definición
Activity	Componente fundamental de Android que representa una pantalla de la interfaz de usuario. Gestion
API Level	Número entero que identifica la versión del framework de Android. La aplicación soporta de API 24 (
APK	Android Package. Formato de archivo binario que contiene todos los recursos, código compilado (DE
Carbohidratos (Carbos)	Macronutriente energético principal. Aporta 4 kcal/g. En el contexto de la app, se mide en gramos y s
CardView	Componente de la librería Material Design que muestra contenido en una tarjeta con elevación y esc
DEX	Dalvik Executable. Formato de bytecode optimizado para la máquina virtual de Android (ART/Dalvik)
FileProvider	Componente de seguridad de Android que permite compartir archivos entre aplicaciones de forma s
Gradle	Sistema de automatización de build utilizado por Android Studio para compilar, testar y empaquetar
Hash (SHA-256)	Función criptográfica unidireccional que produce un resumen de 256 bits. Se usa para almacenar co
IMC	Índice de Masa Corporal. Calculado como $\text{peso(kg)} / \text{altura(m)}^2$. Clasifica el estado nutricional según
Intent	Objeto de mensajería de Android que permite lanzar Activities, iniciar servicios o enviar broadcasts e
Kotlin	Lenguaje de programación moderno, conciso y seguro, desarrollado por JetBrains y adoptado por G
LiveData	Componente de arquitectura de Android que observa cambios de datos de forma reactiva, respetan
Macronutrientes (Macros)	Los tres nutrientes que aportan energía: proteínas (4 kcal/g), carbohidratos (4 kcal/g) y grasas (9 kcal
Material Design 3	Sistema de diseño de Google (también llamado Material You) que proporciona componentes, temas
RecyclerView	Componente eficiente de Android para mostrar listas y cuadrículas de elementos, reciclando las vista
Repository Pattern	Patrón de diseño que abstrae la lógica de acceso a datos, proporcionando una API uniforme indeper
Salt (criptográfico)	Valor aleatorio único añadido a cada contraseña antes del hashing para evitar ataques de diccionario
SecureRandom	Clase de Java que genera números pseudoaleatorios criptográficamente seguros, respaldada por el
SQLite	Sistema de gestión de bases de datos relacionales embebido, sin servidor, que almacena la base de
SharedPreferences	Mecanismo de almacenamiento de pares clave-valor en Android para persistir configuraciones y esta
TDEE	Total Daily Energy Expenditure. Calorías totales que un individuo quema en un día, calculadas como
TextInputLayout	Componente de Material Design que envuelve un EditText añadiendo etiqueta flotante, iconos de ay
TMB / BMR	Tasa Metabólica Basal (Basal Metabolic Rate). Calorías mínimas que el organismo necesita en repo
ViewModel	Componente de arquitectura de Android que almacena y gestiona datos relacionados con la UI, sobr

Anexo C – Esquema SQL Completo (v10)

A continuación se presenta el esquema SQL completo de la base de datos en su versión 10, tal y como aparece en el método onCreate() de **DataBaseHelper.kt**. Incluye las 10 tablas, sus constraints, los índices de rendimiento y los datos semilla (seed data).

```
-- DÍA A IA - Esquema SQLite v10
-- ████████████████████████████████████████████████████████████████████████████

CREATE TABLE IF NOT EXISTS usuarios (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre        TEXT NOT NULL,
    email         TEXT UNIQUE NOT NULL,
    contrasena    TEXT NOT NULL,
    rol           TEXT DEFAULT 'cliente'
                CHECK(rol IN ('cliente', 'entrenador'))
);

CREATE TABLE IF NOT EXISTS perfiles_corporales (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    usuario_id    INTEGER NOT NULL UNIQUE,
    peso_kg       REAL,
    altura_cm     REAL,
    edad          INTEGER,
    sexo          TEXT DEFAULT 'H' CHECK(sexo IN ('H','M')),
    nivel_act     TEXT DEFAULT 'sedentario',
    FOREIGN KEY(usuario_id) REFERENCES usuarios(id)
        ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS objetivos_nutricionales (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    usuario_id    INTEGER NOT NULL UNIQUE,
    calorías      REAL DEFAULT 2000,
    proteínas     REAL DEFAULT 150,
    carbohidratos REAL DEFAULT 200,
    grasas        REAL DEFAULT 65,
    FOREIGN KEY(usuario_id) REFERENCES usuarios(id)
        ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS ejercicios (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre        TEXT NOT NULL UNIQUE,
    grupo_muscular TEXT,
    descripcion   TEXT,
    peso_default  REAL DEFAULT 20.0
);

CREATE TABLE IF NOT EXISTS rutinas (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    usuario_id    INTEGER NOT NULL,
    nombre        TEXT NOT NULL,
    descripcion   TEXT,
    fecha_creacion TEXT DEFAULT (datetime('now')),
    FOREIGN KEY(usuario_id) REFERENCES usuarios(id)
        ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS rutina_ejercicios (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    rutina_id     INTEGER NOT NULL,
    ejercicio_id  INTEGER NOT NULL,
    orden         INTEGER DEFAULT 0,
    reps_obj      INTEGER DEFAULT 10,
    series_obj    INTEGER DEFAULT 3,
    FOREIGN KEY(rutina_id) REFERENCES rutinas(id) ON DELETE CASCADE,
    FOREIGN KEY(ejercicio_id) REFERENCES ejercicios(id)
);

CREATE TABLE IF NOT EXISTS sesiones_entrenamiento (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
```

[illegible]

Nota: El esquema incluye ON DELETE CASCADE en todas las claves foráneas que dependen de la tabla usuarios, garantizando que al borrar un usuario se eliminan automáticamente todos sus datos relacionados sin necesidad de borrados manuales en cascada desde la aplicación.

Anexo B · Referencias y Bibliografía

Referencias Técnicas – Android y Kotlin

- [1] Google. (2024). *Android Developer Documentation*. Recuperado de <https://developer.android.com/docs>
- [2] Google. (2024). *Material Design 3 – Guidelines*. Recuperado de <https://m3.material.io/>
- [3] JetBrains. (2024). *Kotlin Language Reference*. Recuperado de <https://kotlinlang.org/docs/>
- [4] Google. (2024). *Android Room Persistence Library*. Recuperado de <https://developer.android.com/training/data-storage/room>
- [5] Google. (2024). *Android Security Best Practices*. Recuperado de <https://developer.android.com/topic/security/best-practices>
- [6] Android Jetpack. (2024). *Guide to App Architecture*. Recuperado de <https://developer.android.com/topic/architecture>

Referencias – Seguridad y Criptografía

- [7] OWASP Foundation. (2023). *OWASP Mobile Security Testing Guide (MSTG)*. OWASP.
- [8] NIST. (2020). *NIST Special Publication 800-63B: Digital Identity Guidelines*. National Institute of Standards and Technology.
- [9] Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.

Referencias – Nutrición y Fisiología del Ejercicio

- [10] Harris, J.A. & Benedict, F.G. (1919). A biometric study of human basal metabolism. *Proceedings of the National Academy of Sciences*, 4(12), 370–373.
- [11] Mifflin, M.D. et al. (1990). A new predictive equation for resting energy expenditure in healthy individuals. *The American Journal of Clinical Nutrition*, 51(2), 241–247.
- [12] Schoenfeld, B.J. (2010). The mechanisms of muscle hypertrophy and their application to resistance training. *Journal of Strength and Conditioning Research*, 24(10), 2857–2872.
- [13] OMS. (2022). *Encuesta Nacional de Salud España 2022*. Ministerio de Sanidad de España.
- [14] Bompa, T. & Buzzichelli, C. (2019). *Periodization of Strength Training for Sports* (4th ed.). Human Kinetics.

Referencias – Diseño y UX

- [15] Nielsen, J. (1994). *Usability Engineering*. Academic Press.
- [16] Brooke, J. (1996). SUS: A 'quick and dirty' usability scale. In P. W. Jordan et al. (Eds.), *Usability Evaluation in Industry* (pp. 189–194). Taylor & Francis.
- [17] Google Design. (2024). *Material You – Expressive, adaptive, and personal design system*. Recuperado de <https://material.io/blog/announcing-material-you>

Herramientas Utilizadas

- Android Studio Hedgehog 2023.1.1 – IDE principal de desarrollo.
- Git 2.43 + GitHub – Control de versiones y repositorio remoto.
- Figma (Plan gratuito) – Prototipado y diseño de wireframes.
- Notion (Plan gratuito) – Gestión del backlog y documentación de sprints.
- Python 3.11 + ReportLab 4.x – Generación de esta memoria en formato PDF.

- draw.io – Diagramas preliminares de arquitectura y E-R.
- Postman – Prueba de endpoints (en fases exploratorias de diseño).
- SQLite Browser (DB Browser for SQLite) – Inspección y depuración de la BD.